

Design and Evaluation of Ultra-Low Latency Cryptographic Hash Algorithm LLH

**Feng Xiutao, Zeng Xiangyong, Wang Zhangyi, Xu Shengyuan, Xiang Zejun, Wang
Huandong, Tang Ming, Tang Deng, Qin Bo, Gao Bo, Zhang Weiguo, Chen Siwei**

2026.06.25

Contents

1 Algorithm Overview.....	6
2 Algorithm Design Principles	10
2.1 Overall Design Rationale	10
2.2 Design Rationale of the Permutation P	12
2.3 Design Rules for the Components.....	13
2.3.1 The Super S-box S_{24}	13
2.3.2 The S-box S_6	14
2.3.3 Derangement and Linear Transformation	14
2.3.4 Row-wise Cyclic Shift (ShiftRows).....	15
2.3.5 Constant Addition (AddConstant).....	15
2.3.6 Summary	16
3 Properties of Algorithm Components.....	16
3.1 Structural Properties.....	16
3.2 Properties of Basic Components	17
3.2.1 The S-box S_6	17
3.2.2 Derangement	18
3.2.3 Linear Transformation.....	18
3.2.4 Row-wise Cyclic Shift (ShiftRows).....	19
3.2.5 Constant Sequence	19
3.3 Super S-box	20
3.4 Round function.....	20
3.5 Full Diffusion Properties	21
3.6 Algebraic Degree Properties	22
4 Algorithm security analysis.....	26
4.1 Security Analysis of the Overall Structure.....	26

4.1.1 Security Model and Assumptions.....	26
4.1.2 Indistinguishability.....	27
4.1.3 Collision Resistance	30
4.2 Against Length-Extension Analysis	33
4.2.1 Basic Length-Extension Analysis	33
4.2.2 Collision-Based Length-Extension Analysis.....	34
4.3 Preimage-Recovery Analysis	35
4.3.1 SAT-Based Preimage-Recovery Analysis	36
4.3.2 Linearization-Based Analysis of the Equation Systems.....	39
4.4 Second Preimage-Recovery Analysis	41
4.5 Resistance to Collision Analysis	51
4.6 Side-Channel Analysis	53
4.6.1 Target of Side-Channel Analysis	53
4.6.2 Platform Correctness Verification	54
4.6.3 Feasibility Analysis of Template Attacks Against the 24-bit Super S-box	56
4.7 Post-Quantum Security Analysis.....	58
5 Statistical Test for Algorithmic Randomness	61
5.1 Test Instances and Experimental Setup	61
5.2 Baseline Statistical Results	62
5.3 NIST SP 800-22 Statistical Test Results	63
5.4 Avalanche Effect Test Results	65
5.5 Conclusion.....	66
6 Software and Hardware Performance Evaluation	67
6.1 Software Performance Evaluation.....	67
6.1.1 Intel/AMD Platform	67
6.1.2 ARM Platform	84

6.2 Hardware Performance Evaluation	99
6.2.1 Hardware Implementation.....	99
6.2.2 Computational Efficiency	100
6.2.3 Hardware Area (12nm).....	101
6.2.4 Hardware Latency (12nm)	102
6.2.5 Throughput	102
6.2.6 Normalized Throughput-to-Area Ratio	102
6.2.7 Power Consumption	103
6.2.8 Performance Analysis.....	104
7 Algorithmic Innovation and Advantages.....	105
7.1 Algorithmic Innovation	105
7.2 Algorithmic Advantages	106
8 Conclusion.....	107
Acknowledgement.....	109
References	110

1 Algorithm Overview

LLH is a family of cryptographic hash algorithms based on a variant of the sponge construction[1]. It supports input messages of bit length at most 2^{64} and provides four standard output lengths: 256, 512, 768, and 1024 bits. The core component of the algorithm is a permutation function P defined on a $24 \times w$ bit state. The permutation is designed using bit-slicing techniques and realizes nonlinear confusion and full-state diffusion through multiple rounds of iteration. The overall structure of LLH is shown in Figure 1-1.

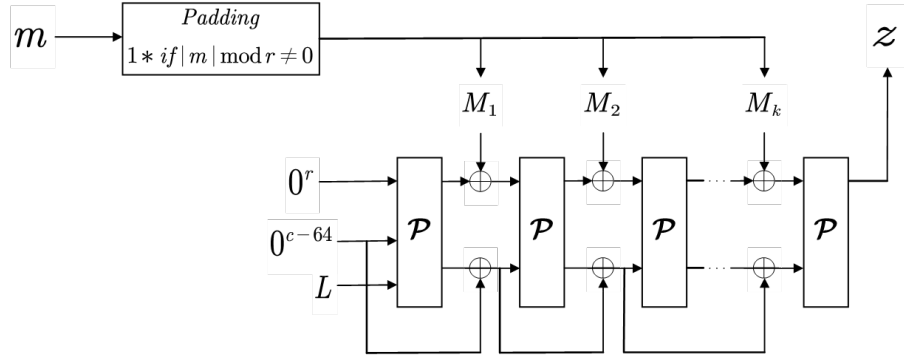


Figure 1-1. Overall structure of the LLH algorithm

LLH provides four instances with different digest lengths, as shown in Table 1-1. Let the word size be w . Then the internal state width is $b=24w$, the rate is $r=8w$, the capacity is $c=16w$, and the digest length is $d=8w$.

Table 1-1. Standard LLH instance parameters

Instance	Word size w	State size b	Output length d	Rate r	Capacity c	Rounds N_r
LLH-256	32	768	256	256	512	16
LLH-512	64	1536	512	512	1024	24
LLH-768	96	2304	768	768	1536	32
LLH-1024	128	3072	1024	1024	2048	40

Let the input message be M and let its length be L bits. The algorithm

first pads the message so that its length becomes an integer multiple of r . If $L \bmod r = 0$, no padding is applied. Otherwise, a number of '1' bits are appended to the end of the message until the total length reaches the smallest integer multiple of r . The padded message is denoted by

$$M' = M_1 \parallel M_2 \parallel \cdots \parallel M_k,$$

where each message block M_i has length r bits. The initial state is then defined as

$$S^{(0)} = 0^{b-64} \parallel L,$$

The algorithm then enters the absorbing phase. For the i -th message block M_i , the current state $S^{(i-1)}$ is first processed by the permutation P . Let $T = P(S^{(i-1)})$. The message block is then XORed into the first r bits of T , namely the rate part, while the last c bits of T are XORed with the last c bits of $S^{(i-1)}$. Equivalently,

$$S^{(i)} = P(S^{(i-1)}) \oplus (M_i \parallel S_{r:b-1}^{(i-1)}), \quad 1 \leq i \leq k.$$

After all message blocks have been absorbed, the algorithm enters the squeezing phase and applies one more permutation

$$S^{(k+1)} = P(S^{(k)}).$$

The final digest is

$$Z = S_{0:r-1}^{(k+1)}.$$

The permutation function P is the core operation of LLH. Its input and output are both $24 \times w$ bit states. Let N_r be the number of rounds. Then P consists of N_r iterations of a round function with the same structure. If the input state is denoted by $X^{(0)}$ and the state after round t by $X^{(t+1)}$, then

$$X^{(t+1)} = \text{RoundFun}(X^{(t)}), \quad 0 \leq t < N_r,$$

The round function consists of three operations: column substitution, row-

wise cyclic shift and constant addition:

- **Column Substitution (SubColumn).** The state is divided into w slices C_j , and each slice is transformed by a 24-bit super S-box $\mathcal{S}_{24}$. Let the input of the super S-box be $(v_0, v_1, \dots, v_{23})$. The transformation $\mathcal{S}_{24}$ is defined as follows:

- 1) First, the 24-bit input is encoded as a 6×4 bit matrix:

$$M_{6 \times 4} = \begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \\ v_{16} & v_{17} & v_{18} & v_{19} \\ v_{20} & v_{21} & v_{22} & v_{23} \end{pmatrix}.$$

- 2) Next, each column of this matrix is transformed by a 6-bit S-box $\mathcal{S}_6$:

$$\mathcal{S}_6(x) = x \oplus (x \ggg 2) \oplus ((x \ggg 1) \wedge (x \ggg 3)) \oplus 0x1B.$$

- 3) The resulting 24 bits are then processed by a derangement, as shown in Table 1-2:

Table 1-2. Derangement in $\mathcal{S}_{24}$

0	1	2	3	derangement →	4	9	18	23
4	5	6	7		8	1	22	15
8	9	10	11		12	17	6	3
12	13	14	15		20	5	10	19
16	17	18	19		0	21	14	11
20	21	22	23		16	13	2	7

- 4) Finally, each row of the permuted matrix is regarded as a 4-bit row vector R_i for $0 \leq i < 6$ and is multiplied on the right by a 4×4 matrix D :

$$D = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

- **Row-wise Cyclic Shift (ShiftRows).** Each row W_i of the state is

regarded as a w -bit word and is cyclically shifted to the left. The shift offset of the i -th word with w is shown in Table, therefore

$$W_i \leftarrow W_i \lll \sigma_i.$$

Table 1-3. Values of Row-wise Cyclic Shift σ_i

i	$w = 32$	$w = 64$	$w = 96$	$w = 128$
0	3	2	2	1
1	16	25	11	16
2	13	20	28	55
3	26	51	53	118
4	8	15	19	10
5	29	40	34	29
6	2	37	57	72
7	23	6	88	11
8	21	36	52	35
9	18	63	73	58
10	31	62	6	105
11	28	33	43	48
12	10	1	5	76
13	15	30	32	103
14	4	31	67	26
15	9	4	14	101
16	7	38	70	5
17	20	5	7	36
18	17	8	48	91
19	30	47	1	42
20	12	19	55	78
21	1	52	94	113
22	6	57	45	44
23	27	34	4	127

- **Constant Addition (AddConstant).** The last word W_{23} of the state is XORed with a round constant:

$$W_{23} \leftarrow W_{23} \oplus RC_t,$$

where RC_t is the w -bit round constant of round t , which is the bits taken from the binary representation of the fractional part of the natural constant e , specifically the bits between the tw -th and $(t + 1)w$ -th

positions.

2 Algorithm Design Principles

2.1 Overall Design Rationale

LLH adopts a variant of the sponge construction. It is structurally simple, friendly to both software and hardware implementation, and is designed to provide high security with low latency. The internal state width is $b = 24w$, the rate is $r = 8w$, the capacity is $c = 16w$, and the output length is $d = 8w$. Under these parameters,

$$\frac{c}{2} = 8w = d.$$

According to the theory of sponge constructions, this parameter choice provides an idealized structural security boundary.

The overall design of LLH follows the principles below:

- **Principle of Structural Simplicity.** LLH uses a variant of the sponge construction. While preserving the advantages of the original sponge construction, it further improves resistance against attacks related to message length. The permutation P is designed using bit-slicing and consists of column substitution, row-wise cyclic shift, and constant addition. In particular, the column substitution is built using a classical SPN structure. This layout makes the overall structure compact and easy to analyze.
- **Principle of Ultra-low latency design.** LLH follows the ultra-low latency design principle throughout the whole design process. This principle is reflected not only in the choice of the overall structure, but also in the selection of all cryptographic components. First, at the core structural level, the permutation P is based on the bit-slice structure. The bit-slice structure is naturally parallelizable and is a mainstream cryptographic design technique

that can support both strong security and low latency. Second, the basic cryptographic components are constructed so that, while preserving the required cryptographic properties, their latency reaches the corresponding mathematical lower bounds. For example, the basic nonlinear component $\mathcal{S}_6$ has branch number 3 and circuit depth 2, reaching the mathematical lower bound for the circuit depth of nonlinear permutation functions. The binary linear transformation D reaches the theoretical branch number upper bound 4, and its circuit depth also reaches the mathematical lower bound 2. The resulting 24-bit super S-box $\mathcal{S}_{24}$ has branch number 5 and circuit depth 4, and has the lowest depth among all known large S-boxes. In addition, the derangement and row-wise cyclic shift do not introduce extra logic depth. Thus the circuit depth of the round function **RoundFun** is 4, which is also among the lowest known circuit depths for round functions.

- **Principle of Implementation friendliness.** All operations in LLH are word-oriented and consist only of low-cost basic operations such as cyclic shifts, bitwise AND, and XOR. Therefore, LLH is convenient for both software and hardware implementation.
- **Principle of Parallelism.** LLH is structurally designed with parallel computation in mind and is especially suitable for SIMD-based implementations. Since the permutation P uses bit-slice technique, logical operations on words naturally process all slices in parallel. Moreover, operations across words and within words are similar, such as XOR, AND, and cyclic shift. These operations can be efficiently implemented using SIMD vector instructions to process multiple words at the same time, thereby improving the efficiency of optimized software implementations.
- **Principle of Transparency.** LLH avoids arbitrary parameters whose

origin is unclear or difficult to justify. It emphasizes openness, verifiability, and reproducibility in both structural and parameter choices. The main components of the algorithm have explicit construction rationales, and the generation rule for round constants is also transparent. This avoids hidden parameters or empirical tuning that could weaken confidence in the overall design.

2.2 Design Rationale of the Permutation P

The permutation function P is the core component of LLH. Its input state is organized as a $24 \times w$ bit matrix, where the i -th row corresponds to a w -bit word W_i , and the j -th column corresponds to a 24-bit slice. Based on bit-slicing, the permutation repeatedly applies column substitution, row-wise cyclic shift, and round-constant addition. The main considerations behind this structure are as follows:

- First, the structure is simple and each operation has a clear purpose. The column substitution provides nonlinear confusion and local diffusion within each column. The row-wise cyclic shift provides diffusion across columns. The round-constant addition breaks possible symmetry and periodic propagation patterns between different rounds.
- Second, the structure supports the low-latency design target. Since row-wise cyclic shift and round-constant addition have negligible delay in hardware logic, the key task for low-latency implementation is to construct a low latency column substitution. For the column substitution, an SPN structure is selected to reduce the difficulty of constructing a large S-box and to facilitate both software and hardware implementation. The S-box layer and the linear layer are chosen to have good cryptographic properties while reaching the corresponding mathematical latency lower

bounds, so that high security and ultra-low latency can be achieved simultaneously.

- Third, the structure is friendly to software, hardware, and SIMD parallel implementation. The whole permutation function P uses only low-cost word-oriented operations such as XOR, AND, and cyclic shift. These operations have low implementation cost and are suitable for SIMD parallelization.

2.3 Design Rules for the Components

2.3.1 The Super S-box $\mathcal{S}_{24}$

The super S-box $\mathcal{S}_{24}$ mainly realizes nonlinear substitution on each slice. Its design follows the principles below:

- It should be a nonlinear permutation;
- It should have very low circuit implementation depth;
- It should provide good confusion and diffusion;
- It should be convenient for software and hardware implementation, with low hardware cost.

Based on these principles, $\mathcal{S}_{24}$ is built using an SPN structure. First, the 24-bit input is encoded as a bit matrix with 6 rows and 4 columns. Then column S-box transformations, a derangement permutation, and row diffusion are applied in sequence. Finally, the transformed matrix is re-encoded to obtain the output of $\mathcal{S}_{24}$.

This layered design makes it possible to impose the low-latency constraint at the level of basic components while also achieving good confusion and diffusion at low cost. It also improves the analyzability of the cryptographic properties.

2.3.2 The S-box $\mathcal{S}_6$

The 6-bit S-box $\mathcal{S}_6$ is the most basic and most important nonlinear component in the permutation P . Its design principles are as follows:

- It should be a nonlinear permutation;
- It should have no fixed points;
- Its circuit implementation depth should reach the mathematical lower bound 2;
- It should have good diffusion, differential uniformity, and nonlinearity;
- It should be simple and low-cost to implement.

To construct a nonlinear S-box satisfying these requirements, we studied a general form of cyclic circuits and obtained a construction method for large nonlinear permutations with diffusion branch number 3 and circuit depth 2. The result is summarized in Theorem 2-1. Compared with the original cyclic-circuit function, this construction provides better diffusion.

Theorem 2-1. *Let $n = 3k$, where $k > 1$ is an integer. Then*

$$f(x) = x \oplus (x \ggg 2) \oplus (x \ggg 1) \cdot (x \ggg 3)$$

is a permutation over $\mathbb{Z}_2^n$.

In the concrete construction used in LLH, $k = 2$ and $n = 6$.

2.3.3 Derangement and Linear Transformation

The derangement and the linear transformation D together form the diffusion part of the column substitution. The role of the derangement is to disrupt row and column correlations in the input matrix without increasing logic depth. Therefore, all positions are required to be displaced, elements from the same original row or column should be distributed across different rows after permutation as much as possible, and the permutation period should be maximized. For the linear transformation D , the main design goal is to

achieve the strongest possible diffusion at very low cost. Accordingly, D is constructed as a 4-bit linear transformation that reaches the optimal branch number and the optimal circuit depth. Combining these two components allows the column substitution to obtain strong local diffusion while preserving low latency.

2.3.4 Row-wise Cyclic Shift (ShiftRows)

The main role of the row-wise cyclic shift is to expand local active patterns formed inside a single column to the whole state. The concrete shift offsets for the rows are mutually distinct, generated by a simple binary polynomial

$$\sigma(x, y) = a_0 + a_1x + a_2y + a_3x^2 + a_4xy + a_5y^2 \pmod{w},$$

and achieve the maximum period. For the 6×4 -bit state matrix corresponding to a single slice, it is required that within each column, at most two words have shift offsets that are congruent modulo 4, while within each row, the shift offsets of all words are pairwise distinct modulo 4, so as to ensure that the permutation structure does not exhibit simple arithmetic-progression properties. This design achieves cross-column diffusion at very low cost, allowing the nonlinearity and local diffusion produced inside the local super S-boxes to spread rapidly to cover the entire state after multiple rounds of iteration.

2.3.5 Constant Addition (AddConstant)

The goal of round-constant addition is to break inter-round symmetry at minimal cost. The round constant is injected only into the last word of the state. Its generation rule is transparent, nonperiodic, and has good apparent randomness. This design avoids introducing a complex constant layer across the whole state while still effectively suppressing symmetric propagation and

short-period trajectories in the state evolution.

2.3.6 Summary

Overall, the components of LLH are not designed to optimize a single metric in isolation. Instead, they are co-designed under the constraints of low latency, low cost, implementation friendliness, and good cryptographic properties. The basic components are kept simple, and stronger local confusion and diffusion are obtained by composition. Through iteration of the whole round structure, this local propagation is further expanded into full-state diffusion. This design philosophy forms the overall architecture of LLH for ultra-low latency implementation.

3 Properties of Algorithm Components

3.1 Structural Properties

At the structural level, LLH uses a variant of the sponge construction. Its internal state width is

$$b = 24w,$$

and its rate, capacity, and output length are

$$r=8w, \quad c=16w, \quad d=8w.$$

Therefore, the idealized structural security boundary corresponding to the capacity is

$$\frac{c}{2}=8w=d.$$

This indicates that, under the current parameter configuration, the structural security boundary matches the target output length. By default, the algorithm performs one squeezing operation after message absorption to obtain the final digest. More generally, the squeezing process may also be parameterized by a number t of squeezing calls, while the total output length

remains $d = 8w$. This structure preserves a clear security boundary for the sponge-variant construction and leaves implementation flexibility for different interface forms.

In addition, the message length L is injected into the initial state before the permutation P is applied. This invalidates traditional attacks related to message length, such as length-extension attacks, and further improves the security of the algorithm.

3.2 Properties of Basic Components

3.2.1 The S-box $\mathcal{S}_6$

The 6-bit S-box $\mathcal{S}_6$ is given by

$$\mathcal{S}_6(x) = x \oplus (x \ggg 2) \oplus ((x \ggg 1) \wedge (x \ggg 3)) \oplus 0x1B.$$

It is a generalization of cyclic-circuit functions. Let the input be $x = x_0 x_1 x_2 x_3 x_4 x_5$ and the output be $y = y_0 y_1 y_2 y_3 y_4 y_5$. The circuit implementation of this function is shown in Figure 3-1.

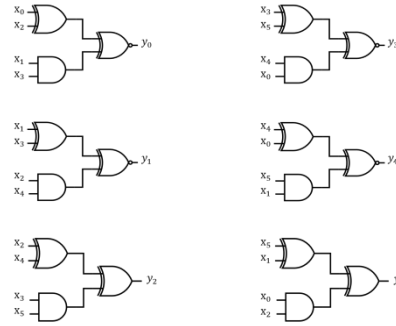


Figure 3-1. Circuit implementation of the S-box function

The S-box $\mathcal{S}_6$ has the following properties:

- Its logic depth is 2, reaching the mathematical lower bound for nonlinear cryptographic permutation functions;
- Its branch number is 3, achieving the best diffusion indicator while keeping the depth optimal;
- Its differential uniformity is 16;

- Its nonlinearity is 16;
- It has no fixed points.

A comparison between the cryptographic properties of $\mathcal{S}_6$ and those of a 5-bit cyclic-circuit function χ of SHA-3 is shown in Table 3-1.

Table 3-1. Property comparison between $\mathcal{S}_6$ and a cyclic-circuit function χ of SHA-3

Item	$\mathcal{S}_6$	χ
Branch number	3	2
Differential uniformity	16	8
Nonlinearity	16	8

3.2.2 Derangement

The derangement disperses element positions in the input matrix so that differences can propagate to other parts of the slice. It has the following properties:

- The arrangement is a derangement;
- The elements in each column are rearranged, and the period reaches the maximum value 6;
- The elements of any row are distributed into different rows after rearrangement.

3.2.3 Linear Transformation

The linear transformation D is a binary matrix. Its circuit implementation is shown in Figure 3-2.

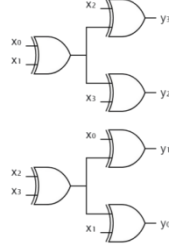


Figure 3-2. Circuit implementation of the linear transformation D

The transformation D has the following properties:

- Its branch number is 4, reaching the optimum for a 4×4 binary matrix;
- Its circuit implementation depth is 2, also reaching the theoretical optimum;
- It requires only 6 XOR gates in implementation.

3.2.4 Row-wise Cyclic Shift (ShiftRows)

The row-wise cyclic shift has the following properties:

- row-wise cyclic shift $w_i = \sigma(x, y)$ is a permutation polynomial over the ring Z_w with respect to two variables (x, y) , where $x = i \gg 2$, $y = i \bmod 4$, and i is the index.
- The permutation period with respect to w reaches the maximum value w ;
- There is no simple arithmetic-progression relation between images.

3.2.5 Constant Sequence

The round-constant sequence is obtained by truncating the binary expansion of the fractional part of the natural constant e (Euler's number). It is transparent, has no significant period, and has good empirical randomness. The constant is injected only into the last word, which has very low cost but is sufficient to break inter-round symmetry.

3.3 Super S-box

The super S-box $\mathcal{S}_{24}$ is a 24-bit-to-24-bit permutation. It has the following properties:

- Its branch number is 5;
- Its circuit implementation depth is 4;
- Each output bit depends on 12 input bits;
- There is no differential pair with nonzero probability in which only the first-row input difference and the output difference are simultaneously nonzero.

The distribution of input-output differential weights for the super S-box is shown in Table 3-2. The distribution indicates that low-weight input-output differential pairs are clearly restricted, while higher-weight propagation occupies the dominant part.

Table 3-2. Differential-weight distribution of $\mathcal{S}_{24}$

Δx	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

3.4 Round function

The single-round function has the following properties:

- Its branch number is 5;
- Its circuit implementation depth is 4;
- Each output bit depends on 12 input bits.

For two rounds of iteration, the diffusion capability is significantly enhanced:

- The branch number increases to 13;
- Each output bit depends on 144 input bits.

This result shows that, after two rounds, local intra-column diffusion rapidly expands to a larger range under the row-wise cyclic shift, and different columns become clearly coupled. In other words, LLH does not rely on a large number of rounds to accumulate diffusion slowly. Instead, significant diffusion growth is already obtained after a small number of rounds.

3.5 Full Diffusion Properties

From the perspective of full-state propagation, all LLH parameter instances reach full diffusion within a small number of rounds. The full-diffusion round numbers for the instances are shown in Table 3-3.

Table 3-3. Full-diffusion round numbers of LLH instances

LLH-256	LLH-512	LLH-768	LLH-1024
4	4	5	5

This result shows that, although a larger word size w increases the state size, the interaction between column substitution and row-wise cyclic shift still allows active patterns to cover the whole state within a small number of rounds. This is especially important for a hash algorithm targeting ultra-low latency: it shows that LLH does not significantly weaken global diffusion efficiency in pursuit of shallow logic depth. On the contrary, the algorithm completes full-state diffusion within 4 to 5 rounds, providing a sound basis for selecting the security margin in the number of rounds.

Overall, the component properties of LLH have a clear layered structure. $\mathcal{S}_6$ provides low-latency basic nonlinearity, and D provides optimal small-scale linear diffusion. $\mathcal{S}_{24}$ integrates the local nonlinearity of $\mathcal{S}_6$ and the local diffusion of D into a super S-box with strong propagation capability. In

the round function, column substitution and row-wise cyclic shift jointly realize rapid diffusion of active patterns across different slices. Through the cooperation of components at different levels, LLH obtains its theoretical foundation for both security and ultra-low latency.

3.6 Algebraic Degree Properties

As a variant of the sponge construction, the LLH algorithm employs multiple permutations designed with an SPN structure, and of use the S-box of the algebraic degree 2. This section will evaluate the growth patterns of the algebraic degrees for the four public permutations of the algorithm in both forward and backward directions based on the division property. The results are shown in Table 3-4. Current methods for evaluating the algebraic degree of SPN structures include formula-based theoretical evaluation methods, division property-based evaluation methods, and others. The results from Reference [2] indicate that the division property-based algebraic degree evaluation method is the most accurate among all evaluation methods. Therefore, the algebraic degree evaluation of the LLH algorithm in this section will adopt the division property-based method.

Let $f, g: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ denote two vectorial Boolean functions, and $h=g \circ f$ denote the composite function of the two. Furthermore, assume the algebraic degree of g is $\text{Deg}(g)=d$, and let M denote the division property propagation model of f , whose objective function is to minimize the output division property when the input division property is fixed. By initially fixing the input division property to d and sequentially increasing its value, let in denote the value of the input division property when the minimum value of the output division property first exceeds d . Then, the evaluated algebraic degree of the composite function is $in-1$. The above method requires modeling and solving

the division property for the public permutations of the LLH algorithm. However, because the permutation widths for the different versions of the LLH algorithm are 768, 1536, 2304, and 3072 respectively, these large widths limit the efficiency of bit-based division property evaluation methods. Therefore, word-based division property propagation will be used subsequently to evaluate the algebraic degree of the LLH algorithm.

For convenience of expression, let E_i denote the i -th round of the LLH algorithm's permutation. First, fix the Hamming weight of the input division property to $in=1$, and construct the word-based division property propagation model $h=I \circ E_1$ (where I represents the identity mapping). M_{E_1} constrains the Hamming weight of the input division property to be in , and constrains the Hamming weight of the output division property to be less than or equal to $out = 1$

If the model M_{E_1} is feasible (has a solution), increment in by 1 and solve it again. If the model is infeasible (has no solution), then the algebraic degree of E_1 is $in-1$. When targeting a two-round permutation, construct $h=E_2 \circ E_1$ and repeat the above process to obtain the algebraic degree of the two-round public permutation. The specific steps are detailed in Algorithm 3-1.

Algorithm Description:

Lines 3-4: Set in as the Hamming weight of the input division property and out as the lower bound of the Hamming weight of the output division property.

Line 6: Construct the model M_{in}^{out} , M_{in}^{out} is a word-level division property propagation model for one round of the public permutation, where the

Hamming weight of the input division property is in , and the Hamming weight of the output division property is less than or equal to out .

Lines 7-13: Solve the model M_{in}^{out} . If the model has a solution, increase the Hamming weight of the input division property in ; if the model has no solution, exit the loop and output the number of rounds and the algebraic degree.

Algorithm 3-1: Word-based Separable Algebraic Degree Evaluation Algorithm for LLH

```

1  :   $in = 2$ 
2  :  for  $round \leftarrow 1$  to  $r$  do
3  :       $out \leftarrow in - 1$ 
4  :       $in \leftarrow in - 1$ 
5  :      while True do
6  :          Construct the model  $M_{in}^{out}$ 
7  :          optimize  $M_{in}^{out}$ 
8  :          if model is feasible
9  :               $in \leftarrow in + 1$ 
10:          else
11:              break
12:          end if
13:      print(  $round, in - 1$  )
16: end for

```

Based on the evaluated algebraic degree, for a permutation with a width of 768 bits, a $14 + 9 = 23$ -round zero-sum distinguisher can be constructed with a complexity of 2^{767} ; for a permutation with a width of 1536 bits, a $15 + 10 = 25$ -round zero-sum distinguisher can be constructed with a complexity of 2^{1534} ; for a permutation with a width of 2304 bits, a $16 + 11 = 27$ -round zero-sum distinguisher can be constructed with a complexity of 2^{2303} ; and for

a permutation with a width of 3072 bits, a $17 + 12 = 29$ -round zero-sum distinguisher can be constructed with a complexity of 2^{3071} .

Table 3-4. Algebraic degrees of LLH permutation iterations

Direction		Forward				Backward			
Parameters		w=32	w=64	w=96	w=128	w=32	w=64	w=96	w=128
Round	$r=1$	2	2	2	2	3	3	3	3
	$r=2$	4	4	4	4	9	9	9	9
	$r=3$	8	8	8	8	27	27	27	27
	$r=4$	16	16	16	16	81	81	81	81
	$r=5$	32	32	32	32	243	243	243	243
	$r=6$	64	64	64	64	614	729	729	729
	$r=7$	128	128	128	128	729	1334	1842	1970
	$r=8$	256	256	256	256	758	1485	2188	2796
	$r=9$	512	512	512	512	765	1523	2275	3003
	$r=10$	682	1024	1024	1024	767	1532	2296	3054
	$r=11$	739	1365	1792	2048	767	1535	2302	3067
	$r=12$	758	1479	2133	2730	767	1535	2303	3070
	$r=13$	764	1517	2247	2958	767	1535	2303	3071
	$r=14$	766	1529	2285	3034	767	1535	2303	3071
	$r=15$	767	1533	2297	3059	767	1535	2303	3071
	$r=16$	767	1535	2301	3067	767	1535	2303	3071
	$r=17$	767	1535	2303	3070	767	1535	2303	3071
	$r=18$	767	1535	2303	3071	767	1535	2303	3071

In addition, in the sponge-based construction variant adopted by the LLH algorithm, during message absorption, the message words are XORed into the first 8 words out of the 24-word state. Therefore, in practical scenarios, only the values of these first 8 words can be freely controlled. When evaluating the algebraic degree of the output with respect to these first 8 state words as variables, note that the S-box in the permutation P has algebraic degree 2, and the quadratic terms are formed by multiplying one bit with another bit at a certain offset. As a result, when treating the first 8 state words as variables,

the algebraic degree of the output after one round is 1. In this case, the algebraic degree results evaluated in Table 3-4 can still be applied, with the only modification being that the corresponding number of rounds is increased by 1.

4 Algorithm security analysis

4.1 Security Analysis of the Overall Structure

This section gives a formal security analysis of the overall LLH structure. LLH differs from the standard sponge construction mainly in two aspects:

1. The initial state is no longer all zero; instead, the message length is embedded in the capacity part.
2. A feed-forward mechanism is added to the iteration of the permutation function P .

In the ideal permutation model, this section shows that the overall LLH structure still satisfies:

1. Pseudorandom indistinguishability (PRF).
2. Collision resistance.

4.1.1 Security Model and Assumptions

The security proof of the overall LLH structure is based on the Ideal Permutation Model (IPM)[3]. This model is a standard framework for proving the security of sponge constructions. It separates the implementation details of the underlying permutation and focuses on the security of the mode itself. To make the proof meaningful, the underlying permutation function P is assumed to be a perfect random permutation with the following three key properties:

- (1) **Bijectivity:** For every input $X \in \{0,1\}^n$, there is a unique output $Y = P(X) \in \{0,1\}^n$, and distinct inputs correspond to distinct outputs.

Conversely, the inverse permutation P^{-1} satisfies $P^{-1}(P(X)) = X$.

- (2) **Uniform randomness:** For any input X that has not been queried before, $P(X)$ is uniformly distributed over $\{0,1\}^n$ and is independent of all previous query inputs and outputs. The output of the inverse permutation P^{-1} satisfies the same uniform randomness property.
- (3) **Oracle accessibility:** The adversary may freely query the oracles P and P^{-1} . Each query returns the corresponding output. However, the adversary cannot obtain the internal computation logic of the permutation and cannot predict the output on unqueried inputs.

4.1.2 Indistinguishability

Proposition 4-1. *LLH is a pseudorandom function (PRF). That is, no polynomial-time adversary $\mathcal{A}$ can distinguish the output of LLH from the output of a truly random function with non-negligible advantage.*

Proof. Define two games. The distinguishing advantage of adversary $\mathcal{A}$ is used to measure the pseudorandomness of LLH:

- (1) Real game (Game₀). The adversary $\mathcal{A}$ inputs an arbitrary message m . The oracle $\mathcal{O}_0$, namely the LLH algorithm, computes and returns the digest Z according to the initialization, absorbing, and squeezing procedures of LLH;
- (2) Ideal game (Game₁). The adversary $\mathcal{A}$ inputs an arbitrary message m . The oracle $\mathcal{O}_1$, namely a truly random function, randomly generates and outputs a binary string Z' with the same length as Z . For repeated queries on the same message m , $\mathcal{O}_1$ returns the same Z' , as is standard for a random function with memory.

The distinguishing advantage is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{ind}} = |\Pr[\mathcal{A}^{\mathcal{O}_0}(1^\lambda)=1] - \Pr[\mathcal{A}^{\mathcal{O}_1}(1^\lambda)=1]|,$$

where λ is the security parameter, $\mathcal{A}^{O_0}$ denotes that the adversary $\mathcal{A}$ has access to the LLH oracle $\mathcal{H}$, and $\mathcal{A}^{O_1}$ denotes that the adversary $\mathcal{A}$ has access to the true random function oracle $\mathcal{R}$. If $\text{Adv}_{\mathcal{A}}^{\text{ind}}$ is a negligible function (i.e., the advantage approaches zero as the security parameter λ increases), then LLH satisfies pseudorandom indistinguishability.

We use a reduction argument. Suppose that there exists a polynomial-time adversary $\mathcal{A}$ that distinguishes Game_0 and Game_1 with non-negligible advantage ϵ . Then one can construct a reduction algorithm RED that uses the distinguishing ability of $\mathcal{A}$ to break the ideality of the underlying permutation P , namely to distinguish P from a truly random permutation. This contradicts the assumption that P is an ideal permutation.

Step 1: Oracle simulation by the reduction

The reduction RED receives access to an unknown oracle $\mathcal{O}$ which is either the ideal permutation P or a truly random permutation $\mathcal{R}_P$. Its goal is to determine, using adversary $\mathcal{A}$, whether $\mathcal{O}$ is P or $\mathcal{R}_P$.

To simulate the LLH oracle for adversary $\mathcal{A}$, RED receives a message query M and proceeds as follows:

- (1) It pads M to obtain $M' = \text{Padding}(M) = M_1, M_2, \dots, M_k$;
- (2) It computes the original message length $L = \text{len}(M)$, encoded as a 64-bit binary string, and initializes the state as $S^{(0)} = 0^r || 0^{c-64} || L$;
- (3) It processes each block M_i ($i = 1, 2, \dots, k$) according to the LLH update rule:
 - Compute the permutation input for the current round: $S^{(i)} = P(S^{(i-1)}) \oplus (M_i || S_{r:b-1}^{(i-1)})$;
 - In each round, RED only passes the inputs and outputs of P ;
 - RED treats P as a black box throughout the process and does not

perform the internal computations of P itself.

- (4) Truncate the first r bits of the final state $S^{(k)}$ to serve as the hash output z , and return z to the adversary $\mathcal{A}$;
- (5) The adversary $\mathcal{A}$ outputs a decision bit (1 if it believes it is accessing the LLH oracle $\mathcal{O}_0$, and 0 if it believes it is accessing the true random function oracle $\mathcal{O}_1$). RED directly adopts this result as its own output to distinguish the type of the unknown oracle $\mathcal{O}$.

Step 2: Reduction of the distinguishing advantage

Consider two cases:

Case 1: If the unknown oracle is $\mathcal{O} = P$, then the oracle simulated by $\mathcal{R}$ is exactly the same as $\mathcal{O}_0$ in Game₀. Therefore, the probability that $\mathcal{A}$ outputs 1 is $\Pr[\mathcal{A}^{\mathcal{O}_0}(1^\lambda) = 1]$.

Case 2: If the unknown oracle is $\mathcal{O} = \mathcal{R}_{\mathcal{P}}$, then in the oracle simulated by $\mathcal{R}$, every state $S^{(i)}$ is a uniformly random string independent of previous queried inputs and outputs, except for the standard consistency constraints of the permutation. Since the padding rule is uniquely decodable, different messages lead to different block sequences and hence different state trajectories. The final output $Z' = S_{0:r-1}^{(k+1)}$ is therefore a uniformly random string, matching the output of $\mathcal{O}_1$ in Game₁. Thus the probability that $\mathcal{A}$ outputs 1 is $\Pr[\mathcal{A}^{\mathcal{O}_1}(1^\lambda) = 1]$.

Therefore, the distinguishing advantage of RED between the permutation P and the random permutation $\mathcal{R}_{\mathcal{P}}$ is

$$Adv_{\mathcal{R}}^{perm} = |\Pr[\mathcal{R}^P(1^\lambda) = 1] - \Pr[\mathcal{R}^{\mathcal{R}_{\mathcal{P}}}(1^\lambda) = 1]| = Adv_{\mathcal{A}}^{ind} = \epsilon.$$

By assumption, P is an ideal permutation, and no polynomial-time algorithm can distinguish it from a random permutation with non-negligible advantage. Hence $Adv_{\mathcal{R}}^{perm}$ must be negligible. This contradicts the

assumption that $\mathcal{A}$ has non-negligible advantage ϵ . Therefore, no such adversary $\mathcal{A}$ exists. $\square$

It remains to note that embedding the message length $L = \text{len}(M)$ in the initial state does not affect indistinguishability. The length L is already known to the adversary, since the adversary chooses the message M . It is used only to distinguish messages of different lengths and does not leak any additional information. It also does not introduce distinguishability of the state. In the ideal permutation model, the randomness of the state is guaranteed by the ideality of P , regardless of whether L is embedded.

4.1.3 Collision Resistance

Proposition 4-2. *The collision security strength of the overall LLH structure is $\min(d/2, c/2)$ bits.*

Proof. We will prove it according to collision scenarios.

The adversary's goal in a collision attack is to construct two distinct messages $M \neq M'$ such that, after padding, absorbing, and squeezing, they yield the same digest. According to the message lengths and block sequences, collision scenarios can be divided into two cases.

Let the padded block sequences of M and M' be $M_1, M_2, \dots, M_k$ and $M'_1, M'_2, \dots, M'_k$, respectively. Let the corresponding initial states be $S^{(0)}$ and $S'^{(0)}$, and let the absorbing-state sequences be $S^{(0)}, S^{(1)}, \dots, S^{(k)}$ and $S'^{(0)}, S'^{(1)}, \dots, S'^{(k)}$. The final outputs are denoted by $Z = S_{0:r-1}^{(k+1)}$ and $Z' = S'_{0:r-1}^{(k+1)}$.

Case 1: Different message lengths ($\text{len}(M) \neq \text{len}(M')$).

If $\text{len}(M) \neq \text{len}(M')$, then their length encodings satisfy $L \neq L'$. According to the definition of the initial state:

$$S^{(0)} = 0^r || 0^{c-64} || L$$

Since $\text{len}(M) \neq \text{len}(M')$, it follows that $S^{(0)} \neq S'^{(0)}$.

Next, we analyze the first state update in the absorbing phase:

$$S^{(1)} = P(S^{(0)}) \oplus (M_1 || S_{r:b-1}^{(0)}).$$

Assume that a collision occurs at this stage, i.e., $S^{(1)} = S'^{(1)}$.

Since P is a permutation, it satisfies the property that $P(X) = P(Y)$ if and only if $X=Y$. Given $S^{(0)} \neq S'^{(0)}$, we have $P(S^{(0)}) \neq P(S'^{(0)})$. The attacker may choose M_1 and M'_1 based on the difference between the outputs of $P(S^{(0)})$ and $P(S'^{(0)})$ such that:

$$M_1 \oplus M'_1 = P(S^{(0)})_{0:r-1} \oplus P(S'^{(0)})_{0:r-1}.$$

This choice effectively cancels out the differential in the r -bit zone (the rate). However, to achieve a full state collision, the attacker must also construct a local collision in the bits ranging from r to $b-1$ (the c -bit zone or capacity). The complexity of a brute-force attack to find such a collision, based on the birthday attack, is $2^{c/2}$. Therefore, for a random permutation P , the security strength against local collisions is $c/2$ bits.

Furthermore, since $S^{(0)} = 0^r || 0^{c-64} || L$, we have:

$$P(S^{(0)})_{b-64:b-1} \oplus L = P(S^{(1)})_{b-64:b-1}.$$

If the attacker attempts to alter L by changing the message length to eliminate the differential in the last 64 bits of the c -zone, this will inevitably change the input to the random permutation P , subsequently altering its output. The random permutation P only needs to satisfy the condition that no fixed points exist within the last 64 bits.

Similarly, in each subsequent round of state updates, the complexity of a brute-force birthday attack remains $2^{c/2}$. Consequently, the security strength

against local collisions for the random permutation P is $c/2$ bits. Therefore, the overall collision resistance of the LLH structure based on the random permutation P is $\min(d/2, c/2)$ bits.

Case 2: Same message length but different blocks ($\text{len}(M) = \text{len}(M')$, $k = k'$, exists i s.t. $M_i \neq M'_i$).

Suppose $\text{len}(M) = \text{len}(M')$, so that $L = L'$ and $S^{(0)} = S'^{(0)}$. Since the padding rule is uniquely decodable, if $M_i \neq M'_i$, then the padded block sequences must differ. Let i be the smallest index such that $M_i \neq M'_i$, while $M_j = M'_j$ for all $j < i$.

First, according to the state update rules of the absorbing phase, it is established that:

- (1) When $j = 0$, $S^{(0)} = S'^{(0)}$;
- (2) For $j < i$, $M_j = M'_j$.

By induction, it can be proven that $S^{(j)} = S'^{(j)}$ for all $j < i$. Consequently, at step $j = i - 1$, we have $S^{(i-1)} = S'^{(i-1)}$.

Next, we analyze the state update in the i -th round:

$$S^{(i)} = P(S^{(i-1)}) \oplus (M_i \parallel S_{r:b-1}^{(i-1)}).$$

Based on the injectivity of the permutation P , where $X \neq Y$ implies $P(X) \neq P(Y)$, we have

$$P(S^{(i-1)}) \neq P(S'^{(i-1)}).$$

In subsequent rounds, since $S^{(i)} \neq S'^{(i)}$ and each state update depends on the preceding state, it follows that $S^{(j)} \neq S'^{(j)}$ for all $j > i$. Thus, the final state satisfies $S^{(k)} \neq S'^{(k)}$, resulting in distinct outputs $Z \neq Z'$. In this scenario, to eliminate the differential and construct a collision, an attacker must find a local collision in the bits ranging from r to $b - 1$ (the c -zone)

for $P(S^{(i)})$. The complexity of a brute-force attack based on the birthday paradox is $2^{c/2}$. Therefore, for a random permutation P , the security strength against local collisions is $c/2$ bits. The random permutation P only needs to satisfy the condition that no fixed points exist within the final c bits.

By considering both scenarios, the upper bound for the total probability of an adversary finding a collision is:

$$\max(2^{-d/2}, 2^{-c/2}),$$

where d is the bit length of the final hash value. The capacity c is designed to ensure the security of the algorithm and does not interact directly with the input message.

Consequently, the overall collision resistance of the LLH structure is $\min(d/2, c/2)$ bits.

□

Specifically, for the four versions—LLH-256, LLH-512, LLH-768, and LLH-1024—the parameter configuration is $c=2d$, which provides ample security redundancy for the capacity zone.

4.2 Against Length-Extension Analysis

4.2.1 Basic Length-Extension Analysis

Length-extension attacks exploit a core weakness of Merkle-Damgård constructions and standard sponge constructions[4]. If the adversary knows the hash value z of a message M , then the all-zero initial state of a standard sponge may allow the adversary to continue the state iteration from the final absorbing state and append an additional message M^* , thereby constructing a valid hash value for an extended message $M||M^*$ without knowing the concrete content of M .

LLH fundamentally blocks this attack path by embedding the message

length L into the initial state:

- (1) **Length dependence.** The extended message $M||M^*$ has length $L' = L + |M^*|$. Its initial state is $S'^{(0)} = 0^r || 0^{c-64} || L'$, which differs from the initial state $S^{(0)} = 0^r || 0^{c-64} || L$ of the original message M ;
- (2) **State non-invertibility.** The underlying permutation P is used as a one-way permutation in the mode, and the feed-forward mechanism is added. The adversary cannot invert the final state $S^{(k)}$ to recover the initial state $S^{(0)}$, and cannot construct an initial state matching L' ;
- (3) **Iteration independence.** The state sequence $S^{(0)}, S^{(1)}, \dots, S^{(k)}$ for the original message differs from the state sequence $S'^{(0)}, S'^{(1)}, \dots, S'^{(k)}$ for the extended message. Therefore, the intermediate results of the original computation cannot be reused to derive the new digest.

4.2.2 Collision-Based Length-Extension Analysis

If an adversary has already found a pair of colliding messages $M_1, M_2, \dots, M_k$ and $M_1', M_2', \dots, M_k'$, then for Merkle-Damgård constructions and standard sponge constructions, the adversary can append the same message M^* to both messages and thereby generate additional collisions under the same iteration rule.

LLH avoids this attack by embedding the message length directly into the initial state. If an adversary appends M^* to an already found collision pair, the initial state $S^{(0)} = 0^r || 0^{c-64} || L$ changes immediately. Recomputing the hashes of $M_1, M_2, \dots, M_k$ and $M_1', M_2', \dots, M_k'$ from the new initial state will yield a collision only with a probability bounded by $\max(2^{-d/2}, 2^{-c/2})$ in the random permutation model.

Therefore, by embedding the message length in the initial state and using a feed-forward mechanism, LLH improves resistance against length-

extension attacks compared to Merkle-Damgård constructions and standard sponge constructions.

4.3 Preimage-Recovery Analysis

This section primarily evaluates whether reduced-round versions of LLH admit preimage-recovery attacks that are more efficient than exhaustive search. Let the output length of LLH be $d = 8w$. The generic complexity of recovering a preimage of an ideal hash function is then $2^d = 2^{8w}$ [5]. Because directly analyzing the security of the full-round algorithm is difficult, we first investigate preimage-recovery methods and their complexities for reduced-round models, and then use the results to assess whether such attacks can be extended to the full-round algorithm.

Let S_0 denote the state after a single message block M_1 has been absorbed, and define the subsequent states as

$$S_1 = f(S_0), \quad S_2 = f(S_1), \quad S_3 = f(S_2),$$

where f denotes one application of the round function, as illustrated in Figure 4-1. The $8w$ bits of the single message block M_1 are XORed into the first eight words of the state, while the remaining 16 words retain the constant values determined by initialization and feedback. As the number of rounds increases, the algebraic degree of S_i with respect to the message variables grows exponentially during the first few rounds; the detailed growth pattern is given in Section 3.6. Accordingly, the preimage-recovery problem changes from solving a linear system into solving a system of higher-degree equations.

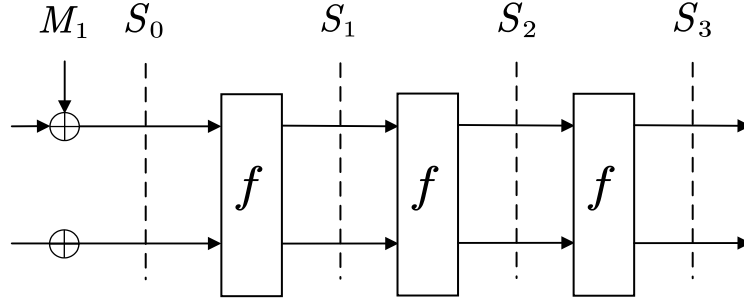


Figure 4-1. State update from S_0 to S_3

The following analysis considers two approaches: CNF modeling followed by SAT solving, and linearization of the corresponding equation systems.

4.3.1 SAT-Based Preimage-Recovery Analysis

The basic SAT-based approach is as follows. We first introduce variables for the intermediate states and establish simple constraints between adjacent intermediate states, as well as constraints relating these states to the input message and output digest. These constraints are then converted into CNF clauses, and the resulting instances are solved using the SAT solver Kissat.

One-round case. A single message block M_1 is XORed into the first eight words of state S_0 , as shown in Figure 4-2. The red blocks denote words affected by M_1 , and the black blocks denote known constants. For each S_6 , only two input bits are affected by the message, while the other four input bits are known constants. It follows directly from the definition of S_6 that each output bit is either a constant or an affine function of the input-message bits. This is illustrated by the 24 output words on the right-hand side of Figure 4-2: black denotes constants, white denotes affine functions of M_1 , and gray denotes condition-dependent values. A gray word is either constant or an affine function of M_1 , depending on the values of the constant input bits.

For a given output digest z , applying the inverse derangement and the

inverse linear transformation D reveals several known words of state T_0 , shown as black blocks in Figure 4-3. Combining Figure 4-2 and Figure 4-3 yields all affine equations for words 1, 4, 18, 22, and 23 of T_0 , together with some of the affine equations for word 15. Since M_1 contains $8w$ bits, on average, each output digest determines $5.5w$ bits of the message M_1 . Therefore, approximately $2.5w$ message bits remain free, and about $2^{2.5w}$ messages map to the same digest z after one round. These messages can be obtained directly by applying Gaussian elimination to the resulting linear system.

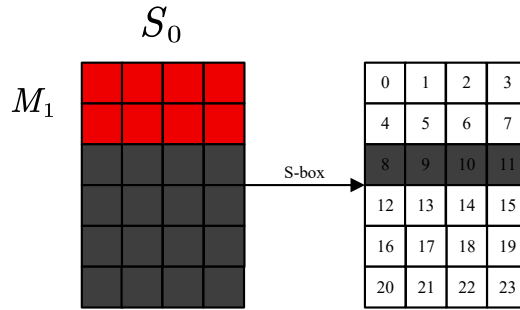


Figure 4-2. State transformation through the super S-box

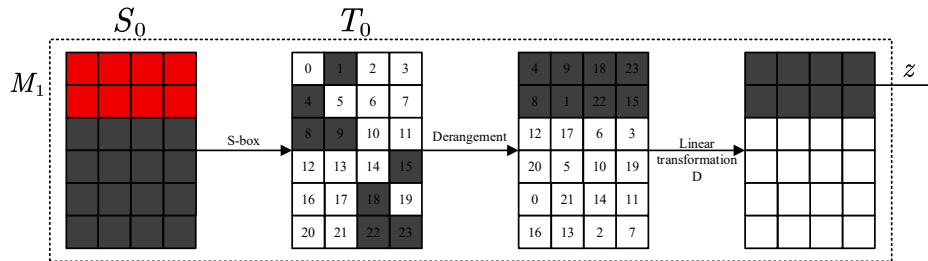


Figure 4-3. One-round iteration of the round function

Two-round case. We first introduce the intermediate-state variables T_0 and T_1 . As in the one-round case, for a given output digest z , applying the inverse derangement and the inverse linear transformation D reveals several known words of state T_1 , shown as black blocks in Figure 4-4. According to the definition of the round function, we establish simple algebraic equations

relating T_0 to M_1 , S_1 to T_0 , and the black blocks of T_1 to S_1 . The equations relating T_0 to M_1 and S_1 to T_0 are linear, whereas those relating the black blocks of T_1 to S_1 are quadratic. After these equations are converted into CNF clauses, the resulting SAT instance contains approximately 2,560 variables and 15,300 clauses. We then ran Kissat on a Dell T640 server equipped with two CPUs, 56 cores, and 512 GB of memory. Within a time-limit of 43,200 seconds, Kissat returns neither SAT nor UNSAT.

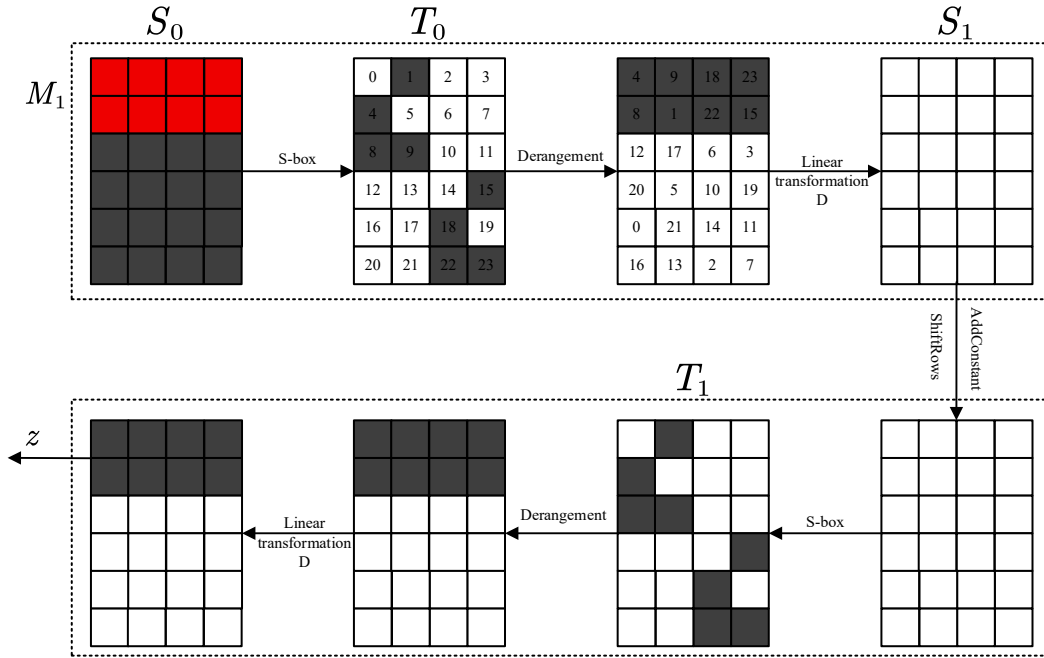


Figure 4-4. Two-round iteration of the round function

Three-round case. Following the same approach as in the two-round case, we first introduce the intermediate-state variables T_0 , T_1 , and T_2 . For a given output digest z , applying the inverse derangement and the inverse linear transformation D reveals several known words of state T_2 , shown as black blocks in Figure 4-5. According to the definition of the round function, we establish simple algebraic equations relating T_0 to M_1 , S_1 to T_0 , T_1 to S_1 , S_2 to T_1 , and the black blocks of T_2 to S_2 . The equations relating T_0 to M_1 , S_1 to T_0 , and S_2 to T_1 are linear, whereas those relating T_1 to S_1

and the black blocks of T_2 to S_2 are quadratic. After these equations are converted into CNF clauses, the resulting SAT instance contains approximately 4,860 variables and 30,650 clauses. Within a time limit of 43200 seconds, Kissat returned neither a SAT result nor an UNSAT result.

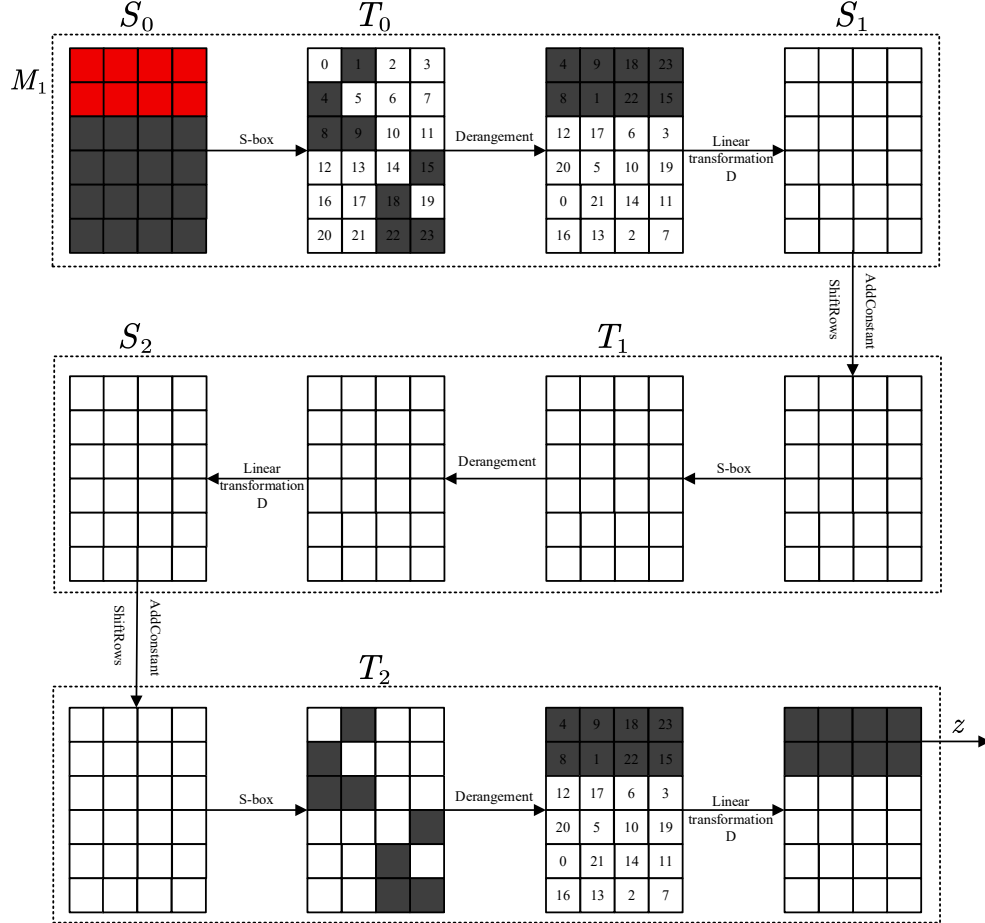


Figure 4-5. Three-round iteration of the round function

For four or more rounds, the resulting CNF instances are larger and more difficult to solve. We therefore believe that solving the preimage-recovery problem using a SAT-modeling approach similar to the one described above is theoretically infeasible for four or more rounds.

4.3.2 Linearization-Based Analysis of the Equation Systems

This section examines the structure of the equation systems and presents

an explicit guess-and-linearize method. The one-round case reduces to solving a linear system, as described in the preceding section, and is therefore omitted here. We focus instead on the two-round case.

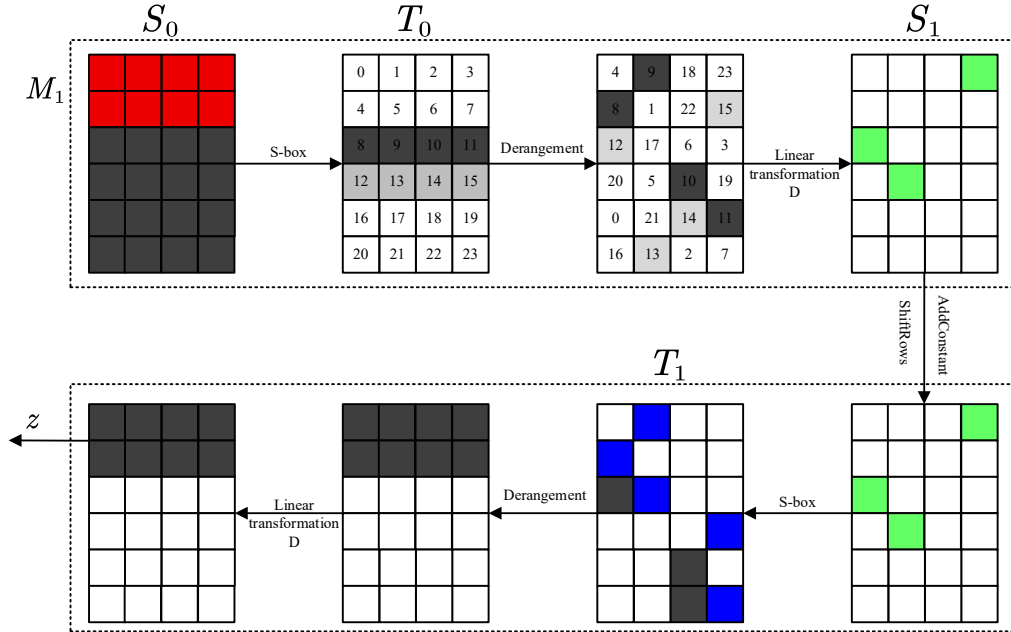


Figure 4-6. Schematic diagram of the two-round permutation structure

We first guess the state words at positions 3, 8, and 13 of S_1 , corresponding to the green blocks in Figure 4-6, for a total of $3w$ guessed bits. Since S_1 is an affine function of the input message M_1 , the guessed values yield $3w$ linear equations. Next, consider the blue block in the first column of state T_1 , whose algebraic equation is

$$y_1 = x_1 \oplus x_3 \oplus x_2 x_4 \oplus 1.$$

Here, x_2 is precisely the third word in the first column of state S_1 , namely the green block in that column. Because its value has been guessed, the equation above reduces to a linear equation.

Next, consider the two blue blocks in the second column of T_1 , whose algebraic equations are

$$y_0 = x_0 \oplus x_2 \oplus x_1 x_3 \oplus 1,$$

and

$$y_2 = x_2 \oplus x_4 \oplus x_3 x_5 \oplus 1.$$

The quadratic terms in these two equations share the variable x_3 . Guessing the value of x_3 , which corresponds to the green block in the second column of state S_1 , therefore reduces both equations to linear equations. Similarly, guessing the green block in the fourth column of S_1 reduces the equations corresponding to the two blue blocks in the fourth column of T_1 to linear equations in the variables of S_1 . We thus obtain linear equations for the five blue words of T_1 . Together with the three guessed words, these provide a total of $8w$ linear equations. Solving this linear system by Gaussian elimination recovers the message M_1 . The complexity of this method is 2^{3w} , which is lower than that of exhaustive search.

We also attempted to extend this method to three rounds. Because the output digest z after three rounds is a degree-4 function of the input message M_1 , linearization requires guessing more than $8w$ bits. The resulting complexity therefore higher than that of exhaustive search. Consequently, we believe that LLH provides strong resistance to preimage-recovery attacks.

4.4 Second Preimage-Recovery Analysis

The goal of a second-preimage attack is, given a message and its hash value, to find another distinct message that produces the same hash output. Since LLH uses a feed-forward sponge structure with block-wise absorption, if only single-block messages are considered, the message space and the output hash-value space are both d bits. In this case, the permutation P in the reduced model can be approximated as a random permutation, and the induced mapping from single-block messages to hash values is very likely to have no nontrivial second preimage. Therefore, low-round second-preimage analysis

mainly considers the two-message-block case.

Let the given two message blocks be m_0 and m_1 . The goal is to find distinct m'_0 and m'_1 such that the final hash value remains unchanged. For a one-round reduced model, after one column substitution, row shift, and constant addition, the output hash value can be expressed as a linear function of the input message variables. Therefore, one can randomly choose $m'_0 \neq m_0$ and then treat preservation of the target hash value as a system of linear equations, solving backward for the corresponding m'_1 . As long as this linear system is solvable, a second preimage for the two-block message can be obtained.

Below, we further give a differential-type second-preimage path for a two-block, two-round reduced model. The analysis is performed on a single 24-bit slice, and sets denote the active row positions in that slice. For example, $\{0,2,4,7\}$ means that nonzero differences appear in rows 0, 2, 4, and 7, corresponding to the 24-bit mask 0x000095. Let

S6 : four parallel 6-bit S-boxes

derangement : a fixed staggered permutation

D : a 4-bit row linear transformation

SuperSbox : $S6 \rightarrow \text{derangement} \rightarrow D$

Round-constant addition does not affect XOR differences, so it can be ignored in differential probability analysis. During absorption of the second block, the attacker can control the low 8-bit difference of the relevant slice through Δm_1 .

To make the subsequent differential path easier to understand, all 6×4 grids in the figures represent a 24-bit slice. Cell positions correspond to row indices, and colored cells indicate positions with nonzero differences. Red denotes the initial propagation path generated by the first-block message

difference Δm_0 ; green and cyan denote Branch A and Branch B formed after the row shift, respectively; white denotes positions with no difference. The probabilities above the arrows come only from S-box differential transitions; derangement, row shift, and the linear transformation D are all deterministic operations and introduce no additional probability.

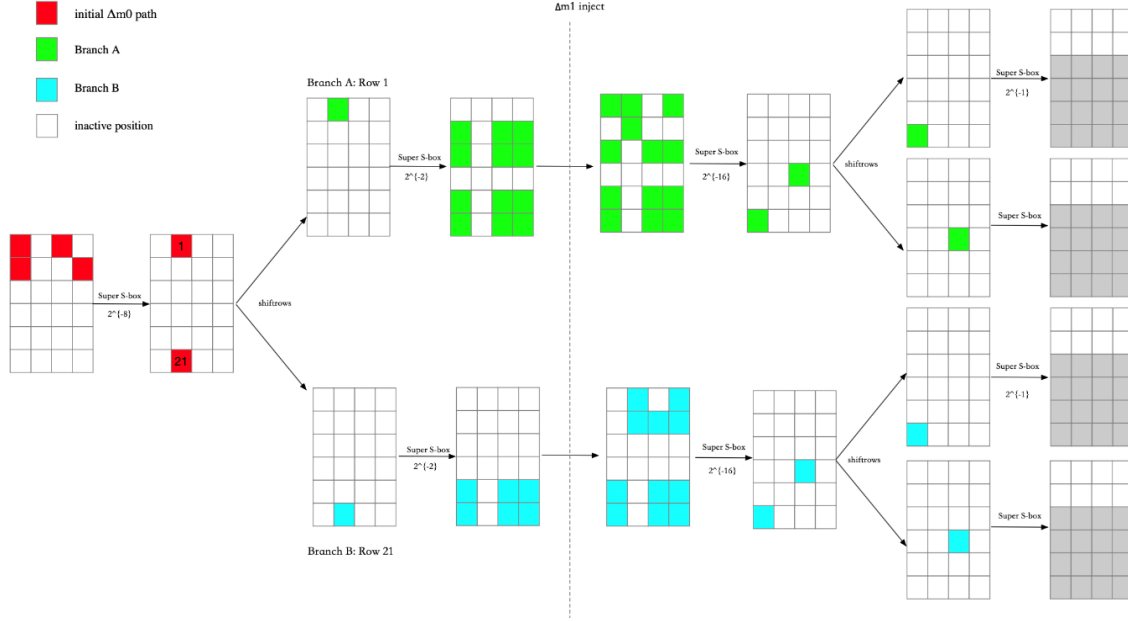


Figure 4-7. Overview of second preimage attack

Figure 4-7 shows the complete differential structure in the two-block, two-round reduced model. In the first stage, the attacker injects an initial difference $\Delta m_0 = \{0,2,4,7\}$ into one column of m_0 , causing it to propagate through one super S-box to $\{1,21\}$ with probability 2^{-8} . In the second stage, the row shift disperses $\{1\}$ and $\{21\}$ into two different columns, forming Branch A and Branch B. The two branches each continue through one super S-box and form fixed high 16-bit difference patterns before absorption of the second block. The third stage on the right side of the dashed line represents the absorption stage of the second message block m_1 , where the attacker chooses the low 8-bit difference of Δm_1 so that both branches enter the same

output pattern {14,20}. Finally, after the row shift, {14,20} is converted into single-bit truncated differences, and the differences at the hash-output positions are eliminated under the subsequent super S-box and linear layer.

In the first stage, choose the initial difference in one column of m_0 :

Delta_in = 0x000095 {0,2,4,7}

S6_out = 0x850094 {2,4,7,16,18,23}

SuperSbox_out = 0x200002 {1,21}

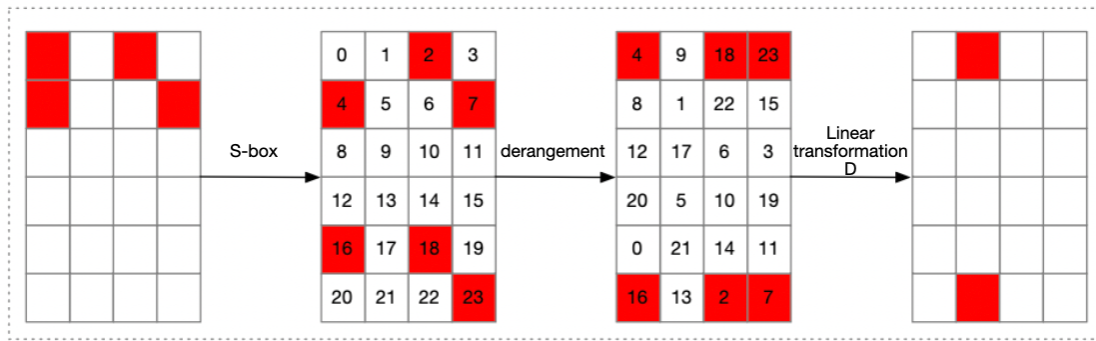


Figure 4-8. Initial difference propagation

The DDT decomposition of the S6 layer is as follows.

Table 4-1. DDT decomposition of initial S-box

Column	Input diff	Output diff	DDT count	Prob.
0	0x03	0x12	4	2^{-4}
1	0x00	0x00	64	1
2	0x01	0x11	16	2^{-2}
3	0x02	0x22	16	2^{-2}

Figure 4-8 gives the exact propagation in the first stage. The input difference {0,2,4,7} first passes through four parallel 6-bit S-boxes. The second grid in the figure marks the S-box output difference {2,4,7,16,18,23}; after the staggered permutation and the linear transformation D, this difference is compressed to {1,21}. Since both derangement and D are

deterministic operations, the probability of this stage is completely determined by the S-box DDT, namely $2^{-4} \times 2^{-2} \times 2^{-2} = 2^{-8}$.

After the first-stage output $\{1,21\}$, the row shift sends row 1 and row 21 into two different columns. If the columns containing the two branches do not collide, the subsequent propagation probabilities can be approximated as multiplicative. For the row 1 branch, choose the following path:

Delta_in = 0x000002 {1}

S6_out = 0x222002 {1,13,17,21}

SuperSbox_out = 0xdd0dd0 {4,6,7,8,10,11,16,18,19,20,22,23}

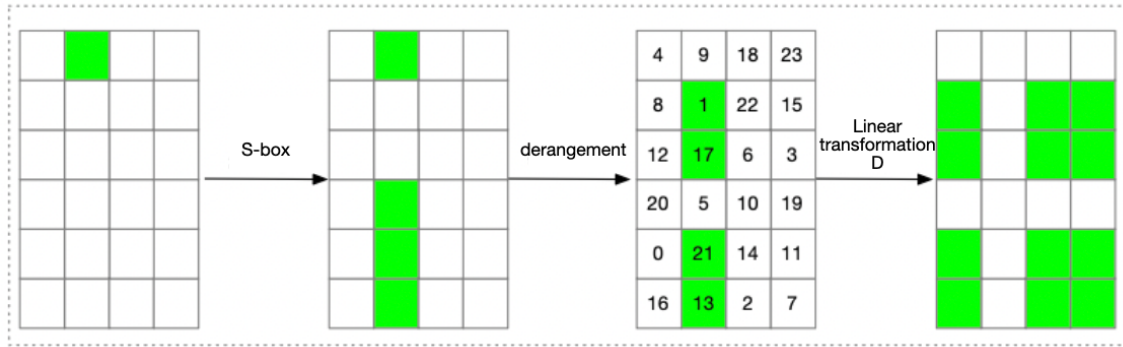


Figure 4-9. Difference propagation of branch A

The only nontrivial column transition in the S6 layer is $0x01 \rightarrow 0x39$, with DDT count 16, corresponding to probability 2^{-2} .

For the row 21 branch, choose the following path:

Delta_in = 0x200000 {21}

S6_out = 0x202000 {13,21}

SuperSbox_out = 0xdd0000 {16,18,19,20,22,23}

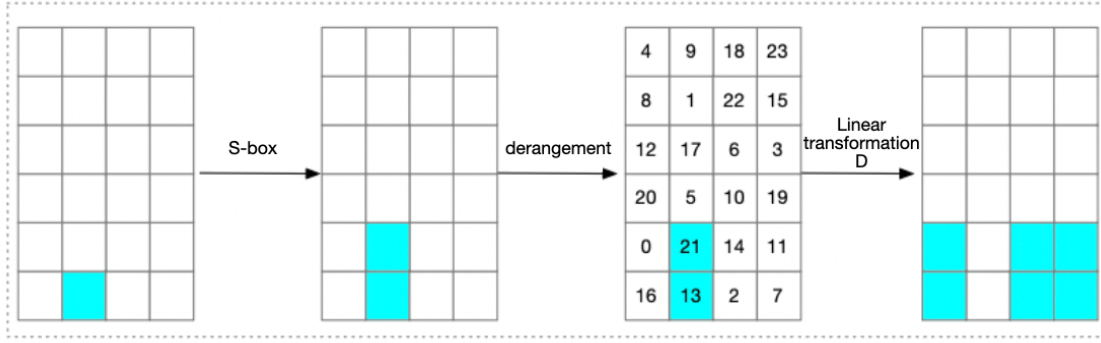


Figure 4-10. Difference propagation of branch B

The only nontrivial column transition in the S6 layer is $0x20 \rightarrow 0x28$, with DDT count 16, corresponding to probability 2^{-2} . Therefore, the combined probability of the two branches in this stage is 2^{-4} .

After entering the absorption of the second message block, the attacker uses Δm_1 to adjust the low 8-bit differences of the two relevant columns so that the complete input differences fall onto favorable paths.

For the row 1 branch, the difference left by the first-block propagation before second-block absorption is $0xdd0dd0$. Choose a low 8-bit difference in the corresponding column of m_1 so that the complete input difference after absorption is

$$\Delta_{in} = 0xdd0d2b$$

Then choose the exact propagation:

$$S6_{out} = 0x1820a4 \{2,5,7,13,19,20\}$$

$$SuperSbox_{out} = 0x104000 \{14,20\}$$

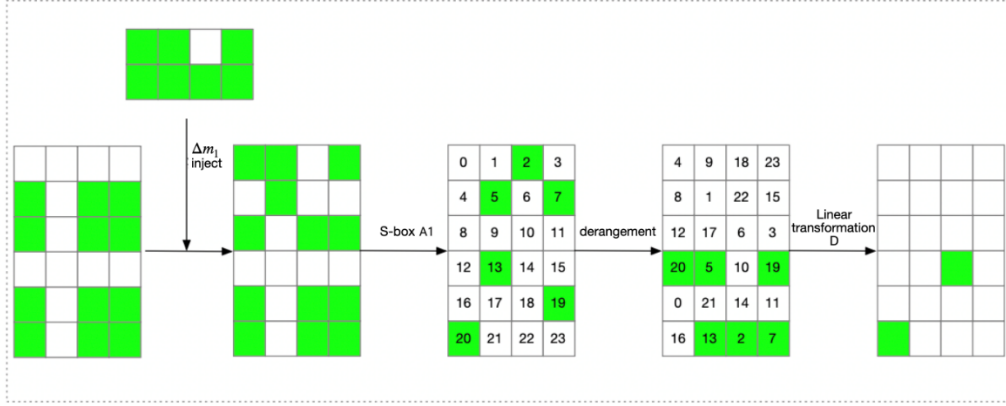


Figure 4-11. Difference propagation of branch A after Δm_1 inject
Its S6-layer DDT decomposition is as follows.

Table 4-2. DDT decomposition of S-box A1

Column	Input diff	Output diff	DDT count	Prob.
0	0x35	0x20	4	2^{-4}
1	0x03	0x0a	4	2^{-4}
2	0x34	0x01	4	2^{-4}
3	0x35	0x12	4	2^{-4}

Therefore, the propagation probability of this branch after second-block absorption is 2^{-16} .

For the row 21 branch, the difference left by the first-block propagation before second-block absorption is 0xdd0000. Choose the low 8-bit difference 0xea = {1,3,5,6,7} in the corresponding column of m_1 so that the complete input difference after absorption is

$$\text{Delta_in} = 0xdd00ea$$

Then choose the exact propagation:

$$\text{S6_out} = 0x1820a4 \{2,5,7,13,19,20\}$$

$$\text{SuperSbox_out} = 0x104000 \{14,20\}$$

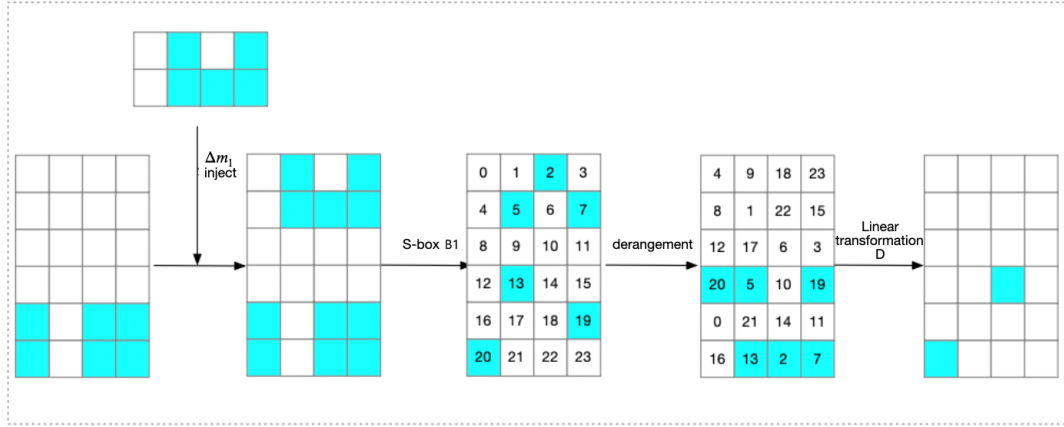


Figure 4-12. Difference propagation of branch B after Δm_1 inject

Its S6-layer DDT decomposition is as follows.

Table 4-3. DDT decomposition of S-box B1

Column	Input diff	Output diff	DDT count	Prob.
0	0x30	0x20	4	2^{-4}
1	0x03	0x0a	4	2^{-4}
2	0x32	0x01	4	2^{-4}
3	0x33	0x12	4	2^{-4}

Therefore, the propagation probability of this branch after second-block absorption is also 2^{-16} .

Figure 4-13 shows the final truncation condition. After adjustment by Δm_1 , both branches output $\{14, 20\}$. After the row shift splits this difference into single-bit differences, the subsequent S-box outputs satisfy the zero-truncation condition at the target hash positions: the differences at $\{1, 4, 8, 9, 15, 18, 22, 23\}$ are 0. The probabilities that these two single-bit differences satisfy the truncation condition after the next S6 layer are computed separately. The single-bit difference at row 14 satisfies this truncation condition with probability 1; the single-bit difference at row 20 satisfies this truncation condition with probability 2^{-1} . Therefore, the

probability of the final truncation stage is 2^{-1} for each branch, and 2^{-2} for the two branches combined. Finally, the deterministic linear layer propagates this truncation condition to the first 8 bits of the hash output, causing the differences at these output positions to vanish.

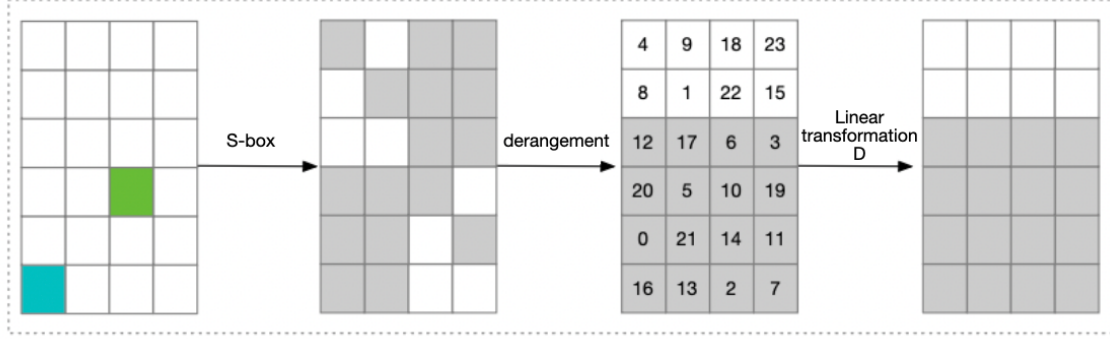


Figure 4-13. Final truncation condition

Multiplying the probabilities of all stages, the total success probability of the path for a fixed initial column is $2^{-8} \times 2^{-2} \times 2^{-2} \times 2^{-16} \times 2^{-16} \times 2^{-2} = 2^{-46}$.

If the attacker can try this type of path at w column positions and the attempts at different columns are approximately independent, then the success probability is about $w \cdot 2^{-46}$, corresponding to an expected complexity of about $2^{46}/w$. For LLH-256, $w = 32$, giving a complexity estimate of 2^{41} .

In summary, in the two-block, two-round reduced model, there exists a structured second-preimage differential path: first, Δm_0 generates two separable single-bit branches; then, Δm_1 adjusts the low 8-bit differences during second-block absorption so that both branches are compressed to $\{14, 20\}$; finally, the single-bit truncation property after the row shift is used to eliminate the hash-output differences. When the path is hit, the attacker can construct $(m_0 \oplus \Delta m_0, m_1 \oplus \Delta m_1)$ as a second-preimage candidate for the original two-block message (m_0, m_1) .

Discussion of Higher-Round Security. The path above shows that, in the two-block, two-round reduced model, the attacker can use the low 8-bit freedom in the second-block absorption stage to locally adjust the branch differences generated by the first-block propagation, thereby constructing a structured second-preimage differential path. However, this path strongly relies on the weak diffusion characteristics of the low-round model. As the number of rounds increases, differences pass through more column substitutions, row shifts, and linear transformations, and active positions rapidly diffuse to more columns and more S-boxes. At that point, controllable low-bit differences in a single message block can no longer easily affect only a small number of local branches, nor can they easily compensate for fixed high-bit differences in multiple columns at the same time. Therefore, the "split first, then locally correct with the low 8 bits of the second block" mechanism in the two-round path no longer holds easily for higher numbers of rounds.

From the perspective of diffusion, differential propagation in the two-round model remains strongly local. The attacker can track the two branches $\{1\}$ and $\{21\}$ and separately choose Δm_1 so that they enter the target pattern $\{14,20\}$. However, after more rounds, row shifts continuously disperse active positions in the same slice into different columns, while the linear layer further expands active differences within columns. As a result, the differential path must simultaneously satisfy transition constraints for an increasing number of S-boxes and must satisfy the final truncation condition across more columns. Compared with the two-round case, the rate at which degrees of freedom grow in the higher-round model is insufficient to offset the growth of constraints caused by diffusion.

The same conclusion can also be drawn from the minimum number of active S-boxes. Let A_R denote the minimum number of S-boxes that must be

activated by any nonzero differential path over R rounds. Since the maximum differential transition probability of each active 6-bit S-box is at most 2^{-2} , the probability of any exact differential path can be upper-bounded by $Pr[\text{path}] \leq 2^{-2 \cdot A_R}$. If the final truncation condition at the hash-output positions must also be satisfied, the actual success probability will only be lower. Therefore, the attack complexity grows at least as 2^{2A_R} . As the number of rounds R increases, the minimum number of active S-boxes rises rapidly, and the differential path probability decreases exponentially. The following sections further support this judgment through evaluation results for the minimum number of active S-boxes at different round counts.

Therefore, the second-preimage path presented in this paper should be understood as a structural analysis of a low-round reduced model, and it cannot be directly extended into an effective attack on full-round LLH. For higher numbers of rounds, stronger diffusion makes locally controllable differences difficult to maintain; the rapid growth in the minimum number of active S-boxes also causes the probability of any exact differential path to decrease exponentially. Combining these two points, LLH can be considered effective at resisting second-preimage attacks at higher numbers of rounds.

4.5 Resistance to Collision Analysis

To evaluate the collision resistance of LLH from the perspective of differential propagation, the core objective is to determine whether high-probability differential paths exist that can be used to construct collisions. To this end, automated methods are employed to search for differential paths with the minimum number of active S-boxes, which are then used to evaluate the security bound against collision attacks.

The differential propagation process is uniformly modeled and automatically searched based on MILP (Mixed Integer Linear Programming) and SAT methods[6]-[7]. The model provides a complete bit-level characterization of the S-box's differential propagation, thereby ensuring the accuracy of the differential reachability description. With the optimization goal of minimizing the number of active S-boxes, it searches for the optimal differential path for a given number of rounds.

In the direct collision search model, constraints are simultaneously applied to ensure that the bitrate portion of the input difference is non-zero and the bitrate portion of the output difference is zero. Furthermore, based on the differential path search, differential filling is completed. However, due to computational complexity limitations, this method can only practically support the search for 1-round collisions. Even when combined with linearized affine subspace techniques, no actual collisions for 2 or more rounds were found.

To further evaluate security, the focus was shifted to establishing a lower bound model for the minimum number of active S-boxes for the permutation itself. This involves directly analyzing the minimum number of active S-boxes required during the differential propagation process. This method can be pushed up to the first 3 rounds. However, as the number of rounds increases, the model size grows rapidly and the computational complexity increases significantly, making it difficult to extend further.

In practical security evaluations, a time-limited search strategy is adopted. Under given computational resource constraints, the minimum number of active S-boxes found within the time limit is treated as an approximately optimal result and compared against the theoretical security bound. Since the maximum differential probability of a single active S-box is 2^{-2} , a differential

path containing N active S-boxes has a probability upper bound of $2^{(-2N)}$. To satisfy a d -bit collision security requirement, the condition $N \geq d/2$ must be met. If the minimum number of active S-boxes found within the time limit reaches or exceeds this threshold, the algorithm is considered to possess sufficient collision resistance in a practical sense.

In the actual search, the MILP execution time is limited to 10 hours. The optimal results obtained from this search are recorded in Table 4-4, where the results for the first three rounds represent global optimal solutions.

Table 4-4. Evaluation results for the number of active S-boxes

$w=32$							
#R	1	2	3	4	5	6	7
#S	1	4	10	24	58	192	266
$w=64$							
#R	1	2	3	4	5	6	7
#S	1	4	10	27	61	299	415
$w=96$							
#R	1	2	3	4	5	6	7
#S	1	4	10	27	61	376	601
$w=128$							
#R	1	2	3	4	5	6	7
#S	1	4	10	28	63	428	834

4.6 Side-Channel Analysis

4.6.1 Target of Side-Channel Analysis

In side-channel analysis of traditional block ciphers such as AES and SM4, the attacker's core objective is usually to recover the fixed round key used in the round function[8]-[10]. The LLH hash algorithm does not contain such an independent key schedule. Nevertheless, in practical cryptographic

applications such as digital signatures, message authentication code (MAC) generation, and key derivation functions (KDFs), the algorithm still processes sensitive information. In these scenarios, the input message M is often formed by concatenating public data with secret information unknown to the attacker, such as a user password or a root key.

Therefore, the side-channel attack target for this hash algorithm must focus on recovering sensitive input message M . According to the sponge-variant design of the algorithm, the input message is padded and divided into message blocks M_i of length equal to the rate r . In the critical absorbing phase, these blocks are sequentially XORed with the first r bits of the internal state. After each XOR operation, a complex permutation function P is immediately applied to mix the state. This XOR-then-permutation mechanism causes many data transitions in the hardware circuit for internal states that contain unknown secret message material. For side-channel analysis, these XOR nodes that continuously absorb fresh message blocks, together with the first subsequent permutation round, form the physical leakage surface.

4.6.2 Platform Correctness Verification

A prerequisite for hardware side-channel analysis (SCA) is deploying the target algorithm on a physical carrier such as an FPGA or ASIC to collect real power or electromagnetic leakage. Therefore, the RTL-level hardware design of the algorithm must first be completed. To this end, the logically verified C implementation `llh_hash_optimize.c` is used as the Golden Model to generate standard internal-state data, which serves as the unique reference for hardware testing.

This hardware design and side-channel evaluation focus on the LLH-512 instance. According to the specification, this instance uses word size $w=64$.

The core of the algorithm is the permutation function P , which consists of multiple iterations of an internal round function. To achieve precise cycle-level control and hardware-resource evaluation, we developed a hardware implementation of the internal round function, namely the Verilog module **iRoundFunc_w64.v**.

In the hardware simulation **Testbench**, we designed a strict word-by-word state comparison logic. The simulation system continuously monitors the output-valid signal **out_valid** at the rising edge of the clock. When valid output is detected, the 64-bit data segment computed by the hardware module is sequentially concatenated into the full state register **got_state**. The platform then automatically compares the completed hardware state with the expected state matrix **expect_mem**, which was precomputed by **llh_hash_optimize.c** and loaded into test memory.

The core verification logic is as follows. If the comparison succeeds, the system prints **[PASS]** and the corresponding test-case number. If data misalignment or a computation error occurs, the system prints **[FAIL]**, together with the incorrect hardware state and the expected value from the C model for debugging.

```
while (i < ROWS) begin
    @(posedge clk);
    if (out_valid) begin
        got_state[i*W +: W] = out_data;
        i = i + 1;
    end
end

if (got_state !== expect_mem[tc]) begin
    fail_cnt = fail_cnt + 1;
```

```

$display("[FAIL] tc=%0d", tc);
$display("  got  = %0384h", got_state);
$display("  exp  = %0384h", expect_mem[tc]);
end else begin
    $display("[PASS] tc=%0d", tc);
end

if (!out_last)

```

After the test platform was built, multiple boundary and random inputs were injected as stimuli. As shown in Figure 4-14, all 8 independent test cases were marked **[PASS]** in simulation. This demonstrates that the **iRoundFunc_w64.v** module is fully consistent with the standard software behavior in both timing scheduling and mathematical functionality.

```

[PASS] tc=1
[PASS] tc=2
[PASS] tc=3
[PASS] tc=4
[PASS] tc=5
[PASS] tc=6
[PASS] tc=7
ALL PASS (8/8)

```

Figure 4-14. Test results

4.6.3 Feasibility Analysis of Template Attacks Against the 24-bit Super S-box

Once the attack target is identified, the next step involves locating nonlinear operations within the algorithm to serve as a focal point for side-channel power modeling. A detailed inspection of the round function (RoundFun) of the permutation P shows that column substitution (SubColumn) is the only nonlinear transformation step in the whole round function. This operation is executed in parallel across w slice columns C_j of the internal state. Each slice column is processed by a 24-bit super S-box S_{24} , a high-

dimensional nonlinear permutation that maps 24-bit inputs to 24-bit outputs.

Currently, the most prevalent and powerful side-channel analysis method is the template attack, also called a Profiling Attack. A template attack is based on multivariate Gaussian models and consists mainly of two phases: template construction and feature matching. During the construction phase, the attacker uses a fully controlled profiling device to characterize the probability distribution of power consumption or electromagnetic radiation for each possible value of the target data, specifically by computing the mean vector and the covariance matrix. Generally, at least 1000 high-quality traces are usually required for each template because physical side-channel collection inevitably contains environmental electronic white noise, oscilloscope quantization errors, and small clock jitters. A large number of repeated samples is essential to suppress these random noises and to ensure sufficient numerical stability and statistical significance when inverting high-dimensional covariance matrices.

However, directly applying profiling attacks to the LLH super S-box is infeasible in terms of sample complexity. The specific theoretical deduction and hardware bottleneck analysis are as follows: first, the data scale for theoretical modeling explodes. To build a fully covering power or electromagnetic model for the 24-bit super S-box $\mathcal{S}_{24}$, all possible inputs must theoretically be enumerated. Since the input width is 24 bits, this requires $2^{24}=16,777,216$ independent profiling templates. Combining this with the requirement of at least 1000 high-quality traces per template, the total number of traces for basic profiling would reach approximately 1.67×10^{10} (close to the 2^{34} level). This is not only excessive in sample complexity, but also creates an overwhelming storage burden for later analysis.

As a trade-off, to avoid building 2^{24} full-space templates, Hamming-

weight-based side-channel energy modeling can be employed. For a 24-bit input message, the Hamming weight has only 25 possible values, from 0 to 24. If templates are built for these Hamming weights, the total number of traces drops to 25000 (25 templates with 1000 traces each). This is feasible on existing hardware acquisition platforms.

Assuming that the 24-bit input message is uniformly random, its Hamming weight follows a binomial distribution. Information entropy can be used to quantify the average recoverable information from a single measurement under this modeling strategy. Initially, the entropy of the 24-bit unknown input message is 24 bits. After measuring the Hamming weight, the average remaining entropy can be computed using the normal-distribution asymptotic approximation, giving an average information recovery of $24 - 20.66 = 3.34$ bits.

Thus, in side-channel analysis of the 24-bit super S-box, Hamming-weight modeling can recover on average about 3.34 bits of effective information from a 24-bit message under reasonable sample and analysis complexity.

In summary, without incorporating any side-channel countermeasures in the hardware design, LLH can theoretically reach the minimum leakage level against the strongest current side-channel attacks considered. To achieve a higher level of security against side-channel attacks in future implementations, hardware-level countermeasures can be appropriately incorporated into the design.

4.7 Post-Quantum Security Analysis

In a quantum-computing environment, the preimage of an ideal d -bit hash function can be searched by Grover's algorithm with a quantum query

complexity of approximately $2^{d/2}$ [11]-[13]. Since LLH has output length $d=8w$, the generic quantum preimage-security targets for LLH-256, LLH-512, LLH-768, and LLH-1024 are 2^{128} , 2^{256} , 2^{384} , and 2^{512} , respectively.

We first discuss one-wayness. That is, after a random message M is selected and its hash value $H(M)$ is computed, it should be difficult for an attacker to output another valid preimage M' satisfying $H(M') = H(M)$. In the quantum model, the attacker is allowed to query the underlying permutation in superposition. For the feed-forward sponge-variasnt mode of LLH, the single-block case can be transformed in the quantum random permutation model into a single iteration with a fixed public capacity part. Specifically, for a single message block, absorption satisfies

$$S^{(1)} = P(S^{(0)}) \oplus (M \parallel S_{r:b-1}^{(0)}).$$

Let the known constant $P(S^{(0)})$ be decomposed as (U, V) . Then the output digest can be written as

$$H_P^{(1)}(M) = R\left(P\left(U \oplus M \parallel V \oplus S_{r:b-1}^{(0)}\right)\right),$$

where R denotes projection onto the rate part of the state. By the corresponding quantum one-wayness result, an attacker making at most q quantum queries has success probability bounded by

$$\Pr\left[OW_{\mathcal{A}}^{(1)}(q) = 1\right] \leq o\left(\frac{q^2}{2^r}\right) + o\left(\frac{q^2}{2^c}\right).$$

The multi-block case can be handled layer by layer using a hybrid argument, yielding the bound

$$\Pr\left[OW_{\mathcal{A}}^{(1)}(q) = 1\right] \leq o\left(\frac{q^2}{2^r}\right) + o\left(\frac{mq^2}{2^c}\right).$$

Since $r = 8w$ and $c = 16w$ in LLH, the dominant term is $O(q^2/$

2^{8w}), corresponding to a quantum preimage-security level of 2^{4w} . The bad-event terms related to the capacity are controlled by $c=16w$ and do not dominate this bound.

At the level of the concrete round function, the quantum analysis follows the reduced-round attack candidates obtained from the classical preimage analysis. The single-block two-round and extremely low-round two-block models can already be solved by classical SAT methods, so they are low-round classically broken models and do not benefit significantly from quantum acceleration as they are already practically broken in the classical setting. The main candidate for the single-block three-round case is the $6w$ partial-linearization route. The classical outer loop needs to enumerate about 2^{6w} fixed patterns. If quantum amplitude amplification is applied only to this outer enumeration, the idealized cost can be reduced to 2^{3w} . For $w = 32$, this outer cost is 2^{96} , formally below the generic quantum baseline 2^{128} for LLH-256. However, this is not yet a complete quantum attack complexity, because each successful outer pattern still requires a local verifier. CNF experiments indicate that even with the correct pattern fixed, the SAT solver (e.g., Kissat) fails to terminate within a reasonable time limit. Consequently, the local verifier cannot be reduced to a low-cost procedure. Therefore, the single-block three-round case should currently be regarded as a quantum-relevant attack candidate, not a complete attack.

For the single-block four-round case, the current partial-linearization route has classical outer complexity at least 2^{12w} . Even with ideal quantum amplitude amplification, this remains at least 2^{6w} , which is still above the generic Grover baseline 2^{4w} . The standard round numbers 16, 24, 32, and 40 are much farther from these reduced-round candidates. Therefore, no

dedicated quantum preimage attack has been found that can extend to the standard-round instances and outperform the Grover baseline. Combined with the mode-level one-wayness analysis, LLH-256, LLH-512, LLH-768, and LLH-1024 achieve generic quantum preimage-security levels of 2^{128} , 2^{256} , 2^{384} and 2^{512} respectively, in the quantum random permutation model.

5 Statistical Test for Algorithmic Randomness

To evaluate the statistical randomness and diffusion properties of the output sequences generated by the LLH hash algorithm, this section conducts a systematic empirical statistical analysis on four standard versions: LLH-256, LLH-512, LLH-768, and LLH-1024. The evaluation framework is structured into three dimensions: first, a fundamental statistical analysis is performed to examine bit-frequency balance and the uniformity of byte distribution; second, the NIST SP 800-22 Statistical Test Suite is employed to conduct standard randomness verification on the output sequences; and third, an avalanche effect test is carried out by introducing single-bit perturbations to the input to assess the algorithm's local diffusion capability. By integrating these three categories of tests, this study provides a multi-faceted assessment of whether the LLH outputs exhibit statistical behaviors consistent with those of an ideal random sequence.

5.1 Test Instances and Experimental Setup

The evaluation instances consist of four standard output versions of the LLH algorithm, namely LLH-256, LLH-512, LLH-768, and LLH-1024. For each variant, test sequences are generated using two distinct input construction strategies:

1. **sequential:** Input messages are constructed in a strictly sequential manner;

2. **random:** Input messages are generated through a stochastic process.

For each specific combination "variant-input" instance, 55 independent 10^6 bits binary sequences are generated to serve as the datasets for formal randomness evaluation. Consequently, the comprehensive NIST statistical testing campaign encompasses a total of eight data groups:

- LLH-256 / sequential
- LLH-256 / random
- LLH-512 / sequential
- LLH-512 / random
- LLH-768 / sequential
- LLH-768 / random
- LLH-1024 / sequential
- LLH-1024 / random

Based on this framework, the proportion of bit "1"s and the byte-level Shannon entropy are further calculated for each output sequence group. Furthermore, avalanche effect tests are conducted for the four variants under input lengths of 16 bytes, 32 bytes, and 64 bytes, respectively. In the avalanche evaluation, 1,000 random samples are employed per group; the output bit-flip ratios are then statistically analyzed following the bit-by-bit inversion of the input message.

5.2 Baseline Statistical Results

First, a baseline statistical analysis was performed on the 8 groups of output sequences generated in the formal testing. The results indicate that the proportions of bit "1" for all 8 groups of sequences are very close to the ideal value of 0.5, with the values ranging from 0.49996116 to 0.50005864, which indicates that the outputs of LLH do not exhibit significant zero-bias or

one-bias phenomena in the sense of overall frequency.

Meanwhile, a byte-level Shannon entropy estimation was performed on the output sequences. The byte entropy for the 8 groups of formal test sequences ranges from 7.99849610 to 7.99854584 bits/byte, which is very close to the 8 bits/byte corresponding to the uniform distribution. This result demonstrates that, from the perspective of byte distribution, the output sequences of LLH do not exhibit any obvious statistical imbalance.

Therefore, at the fundamental levels of frequency balance and distribution uniformity, all four standard versions of LLH exhibit a robust appearance of statistical randomness under both input construction methods.

5.3 NIST SP 800-22 Statistical Test Results

Following the baseline statistical analysis, the NIST SP 800-22 Statistical Test Suite was employed to conduct standard randomness evaluations on the aforementioned eight groups of formal test sequences. The specific test items include:

- Frequency;
- Block Frequency;
- Cumulative Sums;
- Runs;
- Longest Run of Ones;
- Rank;
- Discrete Fourier Transform;
- NonOverlapping Template Matching;
- Overlapping Template Matching;
- Universal;
- Approximate Entropy;

- Random Excursions;
- Random Excursions Variant;
- Serial;
- Linear Complexity.

For a test item with a sample size of 55, the minimum reference pass rate provided in the NIST report is approximately 52/55; for Random Excursions and Random Excursions Variant, as the effective sample size is 44, their minimum reference pass rate is approximately 41/44.

The test results indicate that all eight groups of formal tests successfully completed the entire NIST SP 800-22 evaluation process, with each generating its corresponding `finalAnalysisReport.txt`. From the aggregated results, the fundamental test items—including Frequency, Block Frequency, Runs, Longest Run, Rank, FFT, Universal, Approximate Entropy, Serial, and Linear Complexity—exhibited stable overall performance across all variants and both input construction methods. No systematic anomalies recurring across different versions or input strategies were observed.

Taking the representative results of LLH-256 under the sequential input method as an example, the pass status of its primary test items is as follows:

- Frequency: (54/55);
- Block Frequency: (55/55);
- Cumulative Sums: (53/55) and (52/55);
- Runs: (55/55);
- Longest Run: (54/55);
- Rank: (54/55);
- FFT: (54/55);
- Serial: (54/55) 和 (55/55);

- Linear Complexity: (54/55).

The items mentioned above all reached the corresponding reference thresholds. For Random Excursions and Random Excursions Variant, since the effective sample size was 44, the pass rates for all sub-items reached or exceeded the reference threshold of (41/44), no significant anomalies were observed.

In the representative reports, only a few sub-items of Non-overlapping Template Matching were observed to slightly deviate from the standards. For example, under the sequential input method for LLH-256, one sub-item exhibited a pass rate of (51/55), which is marginally below the reference threshold of (52/55). Given that template-based tests inherently comprise a large number of sub-items, their results typically exhibit high dispersion; thus, minor localized fluctuations in individual sub-items are considered acceptable within empirical statistical testing. More importantly, such fluctuations did not form consistent or recurring anomaly patterns across multiple versions and different input methods, nor were they accompanied by large-scale failures of fundamental test items. Therefore, taken as a whole, no evidence was found to suggest the existence of significant systematic statistical bias in the LLH outputs.

Taking all formal NIST test results into account, it can be concluded that LLH exhibits robust overall performance in terms of statistical randomness.

5.4 Avalanche Effect Test Results

To further investigate the diffusion capability of LLH, this section conducted avalanche effect tests on the four standard versions using input lengths of 16 bytes, 32 bytes, and 64 bytes, respectively. For each input length configuration, 1,000 samples were randomly selected; the output bit-flip ratios

were then statistically analyzed following the bit-by-bit inversion of the input.

The results indicate that the mean bit-flip ratios for the 12 avalanche test groups range from 0.49989435 to 0.50009436, which are highly consistent with the ideal value of 0.5. This demonstrates that when a single-bit perturbation occurs in the input, the LLH output produces an effect where approximately half of the output bits are flipped in a statistical sense, satisfying the fundamental expectations for diffusion in hash functions.

The corresponding standard deviation of the bit-flip ratios ranges from 0.01559049~0.03127503.

From a macro perspective, the standard deviation of the bit-flip ratios exhibits an overall downward trend as the output length increases. This indicates that versions with longer outputs possess more stable diffusion concentration in a statistical sense. Meanwhile, across the three input lengths of 16, 32, and 64 bytes, the mean bit-flip ratios for all four versions remain within a range extremely close to 0.5. This demonstrates that the diffusion capability of LLH is insensitive to variations in input length, exhibiting robust stability.

Consequently, the results of the avalanche tests demonstrate that LLH possesses robust local diffusion capabilities across all standard versions.

5.5 Conclusion

Based on the baseline statistical analysis, the NIST SP 800-22 standard randomness evaluation, and the avalanche effect tests, the following conclusions can be drawn:

First, at the baseline statistical level, the output sequences of the four standard LLH versions—under both sequential and random input construction methods—exhibit a proportion of bit "1s" that is highly close to 0.5, as well

as the byte-level Shannon entropy values are consistently near the ideal value of 8 bits/byte, indicating that the outputs perform well in terms of overall frequency balance and byte distribution uniformity.

Second, in the formal NIST SP 800-22 evaluation, all eight test groups successfully completed the full suite of standard testing procedures. Each group demonstrated stable performance across fundamental items including Frequency, Block Frequency, Runs, Longest Run, Rank, FFT, Universal, Approximate Entropy, Serial, and Linear Complexity, with no consistent anomalies recurring across different versions or input methods. Although minor localized fluctuations were observed in a few sub-items of the template-based tests, these did not constitute a systematic failure pattern. Therefore, the overall results support the conclusion that LLH outputs possess robust statistical randomness.

Finally, in the avalanche effect tests, the mean output flip ratios for all standard versions across various input lengths were extremely close to 0.5. This indicates that LLH possesses excellent diffusion capability in response to single-bit perturbations in the input.

In summary, within the scope of the sample sizes, testing methodologies, and parameter settings employed in tests, none of the four standard versions of the LLH hash algorithm exhibited significant statistical randomness anomalies, suggesting that output sequences demonstrate superior statistical characteristics of randomness.

6 Software and Hardware Performance Evaluation

6.1 Software Performance Evaluation

6.1.1 Intel/AMD Platform

This section evaluates the software performance of the reference and

optimized implementations of the four standard LLH instances on general Intel/AMD x86-64 platforms. The optimizations target the AVX2 instruction path, and tests are conducted in accordance with the official submission standards as required by the Self-Assessment Guidelines for Implementation of New-Generation Commercial Cryptographic Algorithms on x86 Architecture.

The test cases are consistent with those specified in the self-evaluation guideline, covering the eight input lengths S1–S8 defined for the cryptographic hash algorithm, namely 32 B, 128 B, 512 B, 1024 B, 4096 B, 8192 B, 16384 B, and 65536 B.

6.1.1.1 Test Environment and Test Cases

The test platform is an Intel x86-64 physical machine. To ensure reproducibility, the compiler, OpenSSL version, message-size definitions, and CPU binding method were fixed, and all formal measurements were performed on the same logical core. The optimized implementation is compiled following the AVX2 instruction-set implementation path.

Table 6-1. Intel/AMD platform test environment

Item	Specification
Platform type	Physical machine
CPU	Intel(R) Xeon(R) Gold 6258R CPU @ 2.70GHz
Memory	503 GiB
SIMD capability	AVX2
Operating system	Ubuntu 22.04.5 LTS
Kernel version	Linux 6.8.0-107-generic x86_64 GNU/Linux
Compiler	GCC 12.3.0
Build tool	CMake 3.28.3

Test subjects include the reference and optimized implementations of the four standard LLH instances: LLH-256, LLH-512, LLH-768, and LLH-1024. The tests are organized as follows:

1. Functional testing: For each of the eight length points listed in Table 6-2, compute the hash values produced by both the reference and optimized implementations, and compare the results to verify the hash values from both implementations match exactly.

2. Performance testing: For each of the eight length points in Table 6-2, the hash function is executed 100 times. Both the number of elapsed clock cycles and the throughput are recorded for each run, and the arithmetic means are calculated. The average cycle count serves as an indicator of the core computational overhead per single invocation, whereas the average throughput reflects the sustained data-processing capability of the software under continuous load.

3. Test-vector generation: To ensure reproducibility of both functional and performance tests, all input messages of the specified lengths are pseudo-randomly generated using a fixed seed labelled KAT_2_12. This guarantees that the same data set can be recreated whenever needed.

Table 6-2. Intel/AMD platform test cases

Level	Data size
S1	32 Bytes
S2	128 Bytes
S3	512 Bytes
S4	1024 Bytes
S5	4096 Bytes
S6	8192 Bytes
S7	16384 Bytes

6.1.1.2 Selected Optimization Techniques

This subsection describes representative implementation techniques used in the optimized implementation for the AVX2 path on Intel/AMD x86-64 platforms. These optimizations preserve the LLH algorithm definition, input-output semantics, and test-vector results. Their purpose is to improve software execution efficiency on general CPU platforms.

First, separate instantiated implementations are generated for the four standard word sizes $w=32/64/96/128$. This avoids generic runtime parameter dispatch and fixes the round constants, rotation parameters, state loading and storing paths, and internal round-function data paths at compile time. This helps the compiler perform constant propagation, loop unrolling, instruction selection, and register allocation. For hash instances with different word sizes but the same control structure, instance-specific expansion significantly reduces the overhead of generic code and improves instruction locality in the core round function.

Second, the optimized implementation is designed for the AVX2 instruction path. The Boolean operations, word-level cyclic shifts, and state rearrangements in the core round function are parallelized. The LLH round function consists mainly of bitwise XOR, bitwise AND, bitwise NOT, and fixed shifts. These operations have strong data-parallel characteristics and are suitable for mapping to AVX2 instruction paths. The optimized implementation organizes state updates in vectorized form to reduce the number of instructions and memory-access pressure per round, and to improve throughput when multiple state components are updated in parallel.

Third, for byte-aligned input, the optimized implementation uses a dedicated fast processing path. The interface layer first branches according

to whether the message bit length is a multiple of 8. For byte-aligned messages, it enters a specialized whole-byte input-processing path directly. Only when the final part contains a partial byte does it use the general bit-mask and bit-level finalization path. This mechanism reduces buffer management, boundary checks, and control-transfer overhead in short-message scenarios, and is especially beneficial for the S1-S4 length range (32 B to 1024 B).

In addition, the optimized implementation separates the processing of complete message blocks and tail blocks. Complete blocks are processed continuously when the current position is block-aligned, with vectorized whole-block processing when possible. Tail data shorter than one block are handled by constructing a tail block and then performing the final state transformation and output extraction. For short messages not exceeding one block, the general absorbing process is bypassed, and the implementation directly enters tail-block handling and output. This reduces buffer maintenance, data movement, and boundary-check overhead, improving both short-message latency and long-message continuous-processing efficiency.

Finally, to adapt to the multi-level cache and out-of-order execution features of modern server processors, the optimized implementation constrains state layout, temporary-variable reuse, and memory accesses on the critical path. The basic principle is to keep hot state data in registers or within efficient L1-cache access whenever possible, avoiding unnecessary intermediate writebacks and repeated loads. This improves the balance between arithmetic-logic operations and memory access.

In summary, the performance gains of this AVX2-based optimized implementation mainly come from instance specialization, AVX2 vector parallelization, short-message fast paths, block-level processing separation, and optimized state-data organization.

6.1.1.3 Test Results

(1) Functional Consistency

The test-vector outputs produced by the reference implementation and the optimized implementation are identical across all four LLH instances, confirming that the AVX2 optimization path does not alter the algorithmic functional semantics.

(2) Overall Comparison between Reference and Optimized Implementations

Table 6-8 presents a side-by-side comparison of the reference and optimized implementations for the four LLH instances under the representative long-message scenario S8 (65536 B). As shown, the optimized implementation achieves stable and substantial improvements over the reference implementation across all four instances.

Table 6-3. Summary of optimized vs. reference implementation (S8)

Instance	Reference avg. cycles	Optimized avg. cycles	Cycle- count speedup	Reference throughput (MB/s)	Optimized throughput (MB/s)	Throughput speedup
LLH-256	5607558.98	713344.98	7.86x	31.48	245.23	7.79x
LLH-512	4121539.80	655538.76	6.29x	42.79	267.64	6.25x
LLH-768	6970655.82	1603857.26	4.35x	25.21	110.26	4.37x
LLH-1024	5631252.02	1135454.8	4.96x	31.18	154.47	4.95x

Considering all eight length points from S1 to S8, the average performance gains of the optimized implementation are summarized below.

Table 6-4. Average optimization gains across 8 length points (S1–S8)

Instance	Cycle-count speedup	Throughput speedup
LLH-256	7.56x	7.71x
LLH-512	6.19x	6.20x

LLH-768	4.23x	4.21x
LLH-1024	4.86x	4.88x

From the overall results, LLH-256 and LLH-512 exhibit higher relative performance gains, whereas LLH-768 and LLH-1024 achieve significant yet comparatively lower improvements. The lower relative gains for wider-state instances can be attributed to the fact that they require processing more state words and performing more re-arrange operations within a round. Consequently, even though such instances still benefit considerably from the AVX2 path, their relative speedup tends to be lower than that achieved by narrower-state instances.

(3) Detailed Results Across S1–S8

Table 6-5 reports the detailed performance results for all four instances across the eight length points from S1 to S8. For each length point, the table lists both the average cycle counts and the average throughput for the reference and the optimized implementations, respectively.

Table 6-5. S1-S8 performance results on Intel/AMD platforms

Instance	Data size (B)	Reference avg. cycles	Optimized avg. cycles	Reference throughput (MB/s)	Optimized throughput (MB/s)
LLH-256	32	5201.04	728.64	16.6	123.9
LLH-256	128	13229.54	1845.98	25.88	198.58
LLH-256	512	45609.48	5945.76	30.09	232.99
LLH-256	1024	89157	11558.86	30.97	241
LLH-256	4096	352400.82	47397.08	31.11	244.7
LLH-256	8192	697555.44	89903.6	31.71	242.48
LLH-256	16384	1389861.58	180367.22	31.61	244.8
LLH-256	65536	5607558.98	713344.98	31.48	245.23
LLH-512	32	7774.86	1322.2	11.2	66.53
LLH-512	128	11817.56	1902.1	29.04	173.08
LLH-512	512	37278.86	5759.18	38.25	239.73
LLH-512	1024	68813.56	11264.18	39.8	253.4

LLH-512	4096	260875.42	42282.64	41.94	264.91
LLH-512	8192	515594.76	83178.72	42.4	266.7
LLH-512	16384	1032172.62	167250.56	42.82	265.51
LLH-512	65536	4121539.8	655538.76	42.79	267.64
LLH-768	32	19737.36	4919.18	4.35	16.77
LLH-768	128	31162.24	7617.72	11.78	49.39
LLH-768	512	71905.5	17222	19.37	84.25
LLH-768	1024	123043.3	28825.82	22.81	92.69
LLH-768	4096	452046.76	104917.72	24.29	101.73
LLH-768	8192	886817.54	207305.88	24.68	106.42
LLH-768	16384	1776914.9	408400.94	24.76	108.35
LLH-768	65536	6970655.82	1603857.26	25.21	110.26
LLH-1024	32	22242.32	4629.5	4.06	18.72
LLH-1024	128	21811.84	4678.36	15.89	73.85
LLH-1024	512	55382.04	11333.3	25	122.92
LLH-1024	1024	99417.66	20200.16	27.41	137.16
LLH-1024	4096	365359.66	74510.38	30.05	149.84
LLH-1024	8192	710877.16	145595.14	30.7	152.28
LLH-1024	16384	1414198.56	288721.38	30.83	152.15
LLH-1024	65536	5631252.02	1135454.8	31.18	154.47

As shown in Table 6-5, the optimized implementation achieves higher throughput than the reference implementation at every length point and for every instance, and the average cycle counts are consistently and significantly lower than those of the reference across the entire length range. These results indicate that the AVX2 optimizations described in this section deliver robust performance improvements in both short-message and long-message scenarios.

6.1.1.4 Test Conclusion

The comprehensive test results lead to the following conclusions.

First, all four standard LLH instances achieve stable and substantial performance gains from AVX2-based software optimization on general Intel/AMD x86-64 platforms. Relative to the reference implementation, the optimized version not only significantly reduces per-invocation latency in the

short-message range S1–S4, but also sustains consistent speedups in the longer-message range S5–S8.

Second, as illustrated by the representative long-message scenario S8, the optimized implementation achieves cycle-count reduction ratios of $7.86\times$, $6.29\times$, $4.35\times$, and $4.96\times$ over the reference for LLH-256, LLH-512, LLH-768, and LLH-1024, respectively, with corresponding throughput improvements of $7.79\times$, $6.25\times$, $4.37\times$, and $4.95\times$. These results demonstrate that the AVX2 path provides substantial performance enhancements across all four instances rather than marginal fine-tuning.

Third, from the averaged results over the eight length points S1–S8, LLH-256 and LLH-512 exhibit the most prominent gains, while LLH-768 and LLH-1024 maintain optimization factors exceeding $4\times$ in average cycle counts. This indicates that the LLH round function and data path exhibit good SIMD vectorization compatibility, and that the optimized implementation can significantly improve software execution efficiency while preserving full algorithmic semantics, interface behavior, and test-vector consistency.

In summary, the results presented in this section demonstrate that LLH exhibits a good level of optimization potential under AVX2-based software implementations on general Intel/AMD x86-64 platforms.

6.1.1.5 Supplementary Notes on the AVX-512 Optimization Path

In addition to the AVX2 optimization above, LLH was also tested on an AVX-512 based wider vector path in the same environment. This path preserves full algorithmic consistency in definition, I/O semantics, and test vectors. The optimization strategy employs AVX-512 instruction set to enable parallel state organization under wider registers, vectorized Boolean and permutation operations, and instruction-level compression for both short and full-block processing. The following results supplement the AVX-512

performance evaluation and serve as a reference for AVX-512-capable platforms. In this section, the term “optimized implementation” refers exclusively to the AVX-512 based version.

The test objects include the reference and AVX-512 optimized implementations of LLH-256, LLH-512, LLH-768, and LLH-1024. To evaluate software efficiency under the same output length, the following baseline algorithms are used:

- LLH-256 compared with SHA-256;
- LLH-512 compared with SHA-512;
- LLH-768 compared with SHAKE256-768;
- LLH-1024 compared with SHAKE256-1024.

The software results for SHA-256, SHA-512, SHAKE256-768, and SHAKE256-1024 are all from OpenSSL 3.0.2. It should be noted that all baseline comparisons below are used only for software-efficiency analysis under the same output length. They do not constitute cryptographic equivalence comparisons.

Two types of test results are used. The first type, Test1 to Test4, reflects overall performance under several typical message sizes. The second type, Test5, describes fine-grained performance variation with input length in the short-message range. The former uses 100 repeated measurements and reports the average; the latter independently measures each input-length point 30 times and reports the average.

Table 6-6. Intel/AMD platform test cases for AVX-512 optimization

ID	Data size	Meaning
Test1	1B-512B short-message set	Processes each message length from 1B to 512B once and reports the whole short-message interval as one result.

Test2	1KB	Reports the performance for a 1KB message.
Test3	1MB	Reports the performance for a 1MB message.
Test4	1GB	Reports the performance for a 1GB message.
Test5	0B~512B	Reports the performance for every input length from 0B to 512B separately.

The metrics used in this section are execution time, throughput, optimization speedup, and relative difference from the baseline. It is important to distinguish Test1 and Test5: Test1 reflects the overall processing capability over the short-message interval and can be compared side by side with the 1KB, 1MB, and 1GB results; Test5 describes local performance variations at different input lengths in the 0B-512B range. They correspond to different observations.

(1) Reference implementation versus optimized implementation.

Table 6-7 summarizes the performance of the reference and optimized implementations of the four LLH instances on the Intel/AMD platform. Overall, the optimized implementation obtains stable performance gains over the reference implementation.

Table 6-7. Comparison between AVX-512 optimized and reference implementations

Instance	Data group	Reference latency (ms)	Optimized latency (ms)	Reference throughput (MiB/s)	Optimized throughput (MiB/s)	Speedup
LLH-256	1~512B	3.615436	0.442468	34.685116	283.464505	8.171068
LLH-256	1KB	0.024437	0.002921	40.004304	334.93651	8.365971
LLH-256	1MB	24.146783	2.83776	41.442178	352.603499	8.5091
LLH-256	1GB	24769.77576	2897.485173	41.341017	353.411662	8.548715
LLH-512	1~512B	2.286199	0.56015	54.789275	223.654649	4.081405
LLH-512	1KB	0.014116	0.003259	70.073789	305.212601	4.33139
LLH-512	1MB	13.363578	3.063043	74.836982	326.507711	4.362844
LLH-512	1GB	13738.38189	3136.134572	74.538194	326.51841	4.380674
LLH-768	1~512B	7.609766	1.450457	16.459571	86.36024	5.246461

LLH-768	1KB	0.042873	0.008033	22.781962	121.583493	5.337109
LLH-768	1MB	39.018024	7.305698	25.631864	136.892905	5.340766
LLH-768	1GB	40071.19708	7475.691516	25.554606	136.978336	5.360199
LLH-1024	1~512B	6.012693	1.156127	20.835207	108.358077	5.20072
LLH-1024	1KB	0.030114	0.006017	32.641641	167.183567	5.00482
LLH-1024	1MB	27.337303	5.259274	36.586926	190.196544	5.197923
LLH-1024	1GB	29669.99969	5408.945536	34.513521	189.317395	5.485357

Figure 6-1 gives the throughput results of the four LLH instances under four message-size categories. The throughput of the AVX-512 optimized implementation is clearly higher than that of the reference implementation for every instance and message size.

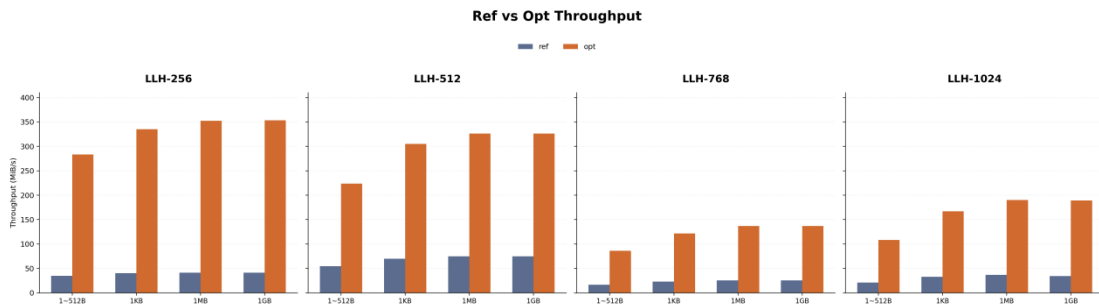


Figure 6-1. Throughput comparison between reference and AVX-512 optimized implementations

Figure 6-2 compares reference and optimized implementations in terms of execution time. The optimized implementation has significantly lower latency for every instance.

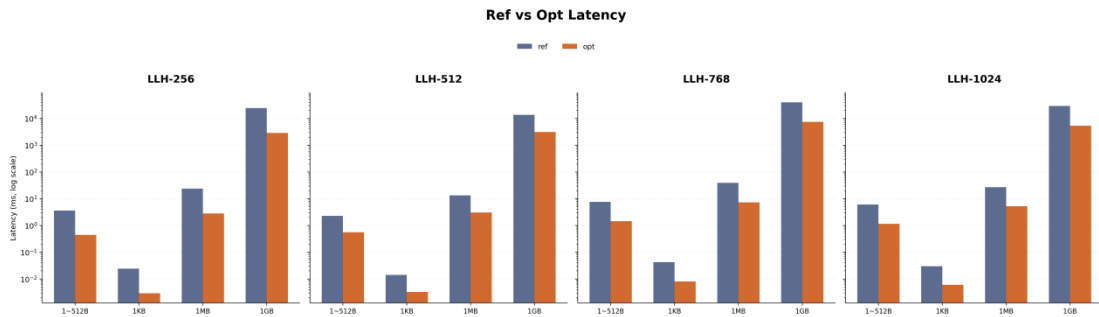


Figure 6-2. Latency comparison between reference and AVX-512 optimized implementations

Figure 6-3 shows the optimization speedup under different message sizes. The speedup of each instance varies only moderately across message sizes, indicating that the optimization effect is stable.

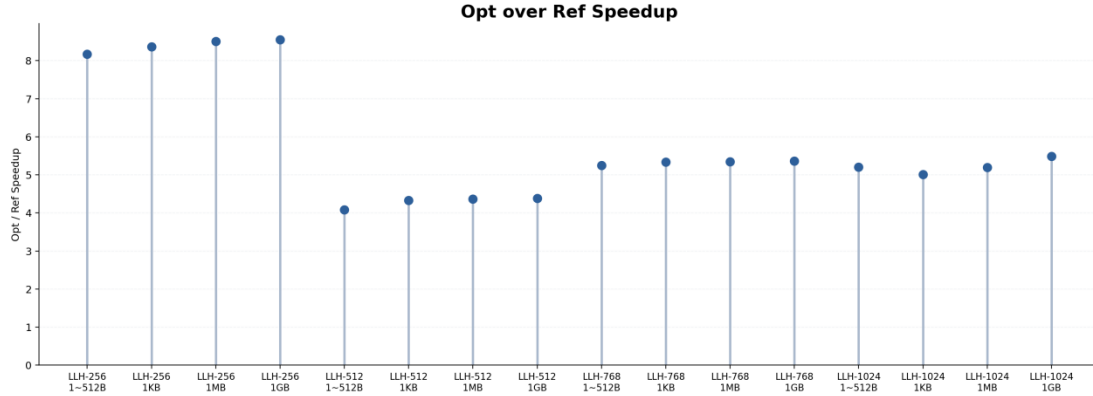


Figure 6-3 Speedup of AVX-512 optimized implementation over reference implementation

(2) Fine-grained short-message results.

Figure 6-4 shows throughput as a function of input length in the short-message interval. Overall, the throughput curve of the optimized implementation is above that of the reference implementation.

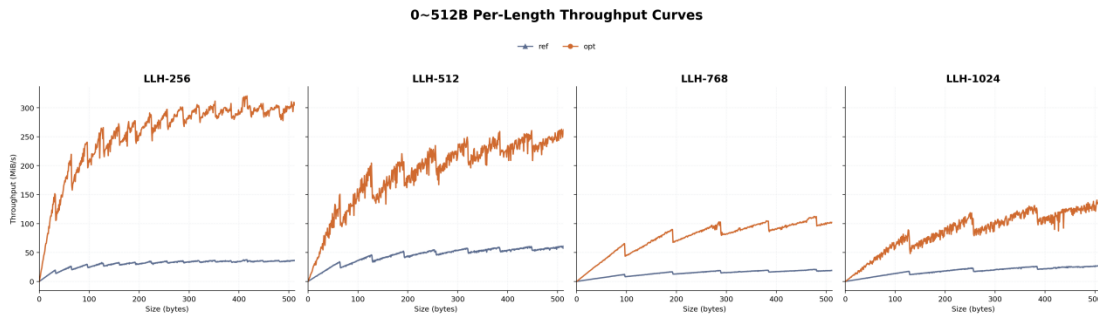


Figure 6-4. Throughput curve for each input length from 0B to 512B

Figure 6-5 shows the latency variation in the short-message interval on a logarithmic scale. Overall, the latency curve of the optimized implementation is below that of the reference implementation.

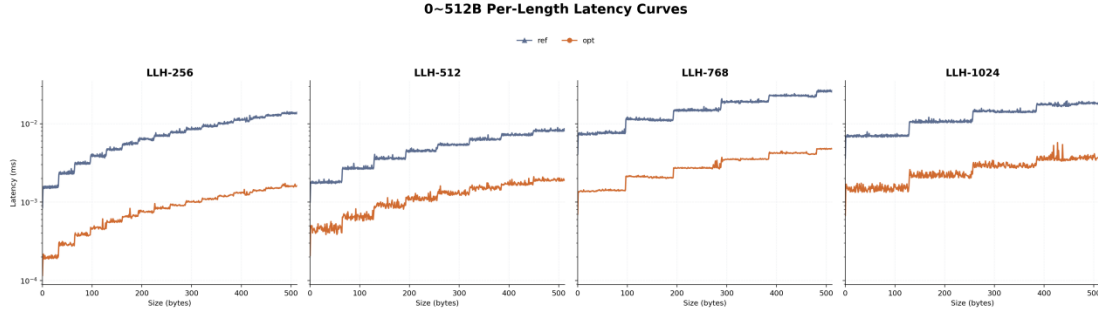


Figure 6-5. Latency curve for each input length from 0B to 512B

(3) Comparison with baseline algorithms.

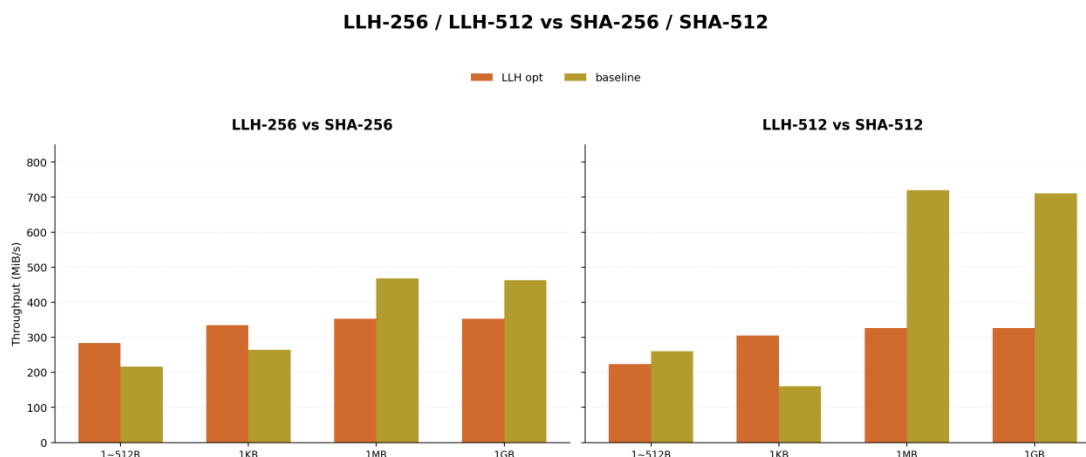
Table 6-8 summarizes the software-efficiency position of the AVX-512 optimized implementation relative to the baseline algorithms. Different instances show different relative advantages under different message sizes.

Table 6-8. Summary comparison between AVX-512 optimized implementation and baseline algorithms

Instance	Data group	Optimized latency (ms)	Baseline latency (ms)	Optimized throughput (MiB/s)	Baseline throughput (MiB/s)	Optimized relative to baseline
LLH-256	1~512B	0.442468	0.583416	283.464505	216.180759	31.12%
LLH-256	1KB	0.002921	0.003694	334.93651	264.465041	26.65%
LLH-256	1MB	2.83776	2.138659	352.603499	468.234064	-24.70%
LLH-256	1GB	2897.485173	2210.407338	353.411662	463.330884	-23.72%
LLH-512	1~512B	0.56015	0.480522	223.654649	260.858491	-14.26%
LLH-512	1KB	0.003259	0.0061	305.212601	160.721032	89.90%
LLH-512	1MB	3.063043	1.390003	326.507711	719.530901	-54.62%
LLH-512	1GB	3136.134572	1441.571258	326.51841	710.360735	-54.03%
LLH-768	1~512B	1.450457	0.680698	86.36024	184.029642	-53.07%
LLH-768	1KB	0.008033	0.003326	121.583493	294.91693	-58.77%
LLH-768	1MB	7.305698	2.629677	136.892905	380.336257	-64.01%
LLH-768	1GB	7475.691516	2708.369742	136.978336	378.093105	-63.77%
LLH-1024	1~512B	1.156127	0.654605	108.358077	191.423142	-43.39%
LLH-1024	1KB	0.006017	0.009446	167.183567	103.505808	61.52%

LLH-1024	1MB	5.259274	2.589454	190.196544	386.387013	-50.78%
LLH-1024	1GB	5408.945536	2762.050165	189.317395	370.741501	-48.94%

Figure 6-6 compares the throughput of LLH-256 and LLH-512 with the corresponding SHA-256 and SHA-512 baselines. LLH-256 is advantageous for short messages and 1KB messages, while LLH-512 exceeds its baseline only at 1KB.



**Figure 6-6. Throughput comparison of LLH-256/LLH-512
with SHA-256/SHA-512**

Figure 6-7 shows the percentage difference in throughput between the optimized implementation and SHA-256/SHA-512 for each input length from 1B to 512B. LLH-256 exceeds the baseline at most length points. The relative position of LLH-512 varies more obviously with input length, but is overall above the baseline. The horizontal axis starts from 1B rather than 0B because the figure shows percentage throughput difference. For 0B input, both the optimized implementation and the baseline have throughput 0, so the relative difference is undefined.

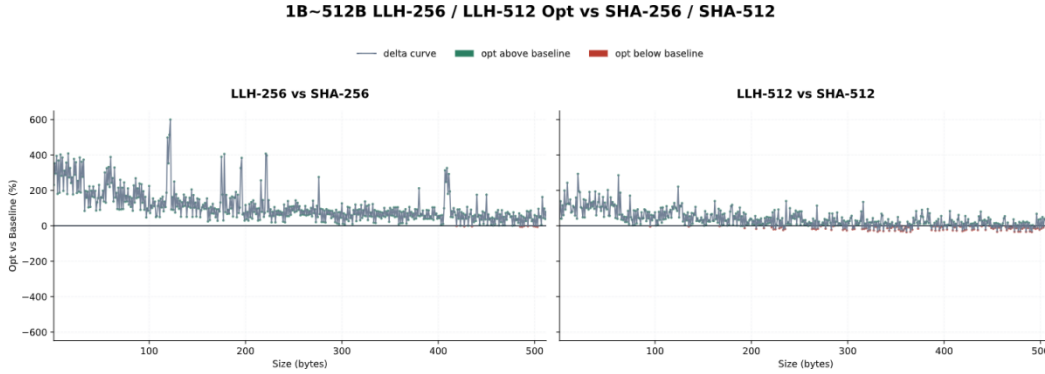
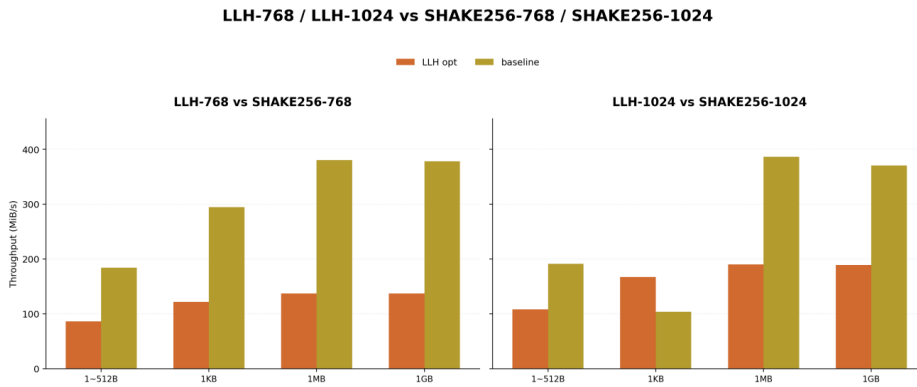


Figure 6-7. Throughput difference of optimized LLH-256/LLH-512

Figure 6-8 compares LLH-768 and LLH-1024 with the corresponding SHAKE256 baselines. LLH-768 is lower than the baseline for all four message sizes, while LLH-1024 exceeds its baseline only at 1KB.



**Figure 6-8. Throughput comparison of LLH-768/LLH-1024
with SHAKE256-768/SHAKE256-1024**

Figure 6-9 shows the percentage difference of throughput between the optimized implementation and SHAKE256-768/SHAKE256-1024 for each input length from 1B to 512B. LLH-768 is mostly below the baseline, while LLH-1024 approaches or exceeds the baseline only over some length intervals. The horizontal axis starts from 1B rather than 0B for the same reason: the relative percentage difference is undefined at 0B input.

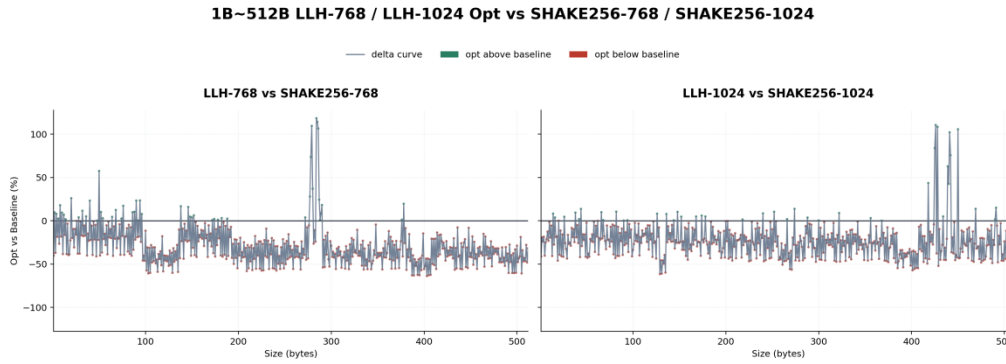


Figure 6-9. Throughput difference of optimized LLH-768/LLH-1024

Figure 6-10 summarizes the throughput percentage difference between the optimized implementation and the baseline algorithms. Positive gains are mainly concentrated in short-message and 1KB cases for LLH-256, the 1KB case for LLH-512, and the 1KB case for LLH-1024.

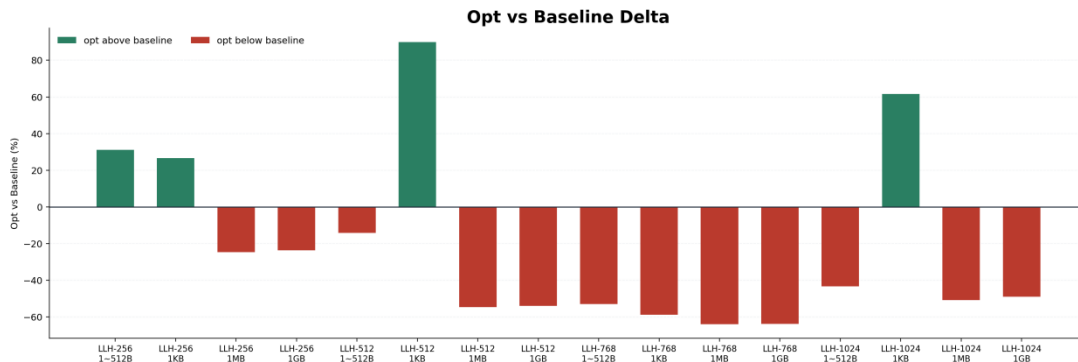


Figure 6-10. Percentage throughput difference of optimized

Figure 6-11 gives the throughput results of the reference implementation, optimized implementation, and baseline algorithms. The optimized implementation clearly improves over the reference implementation, while its advantage over the baselines depends on the instance and message size.

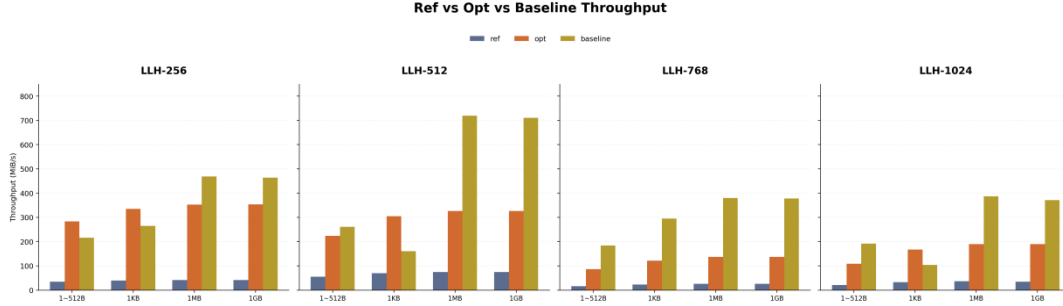


Figure 6-11. Throughput comparison among reference implementation, optimized implementation, and baseline algorithms

All baseline comparisons in this section are performed under the same output-length criterion: LLH-256 is compared with SHA-256, LLH-512 with SHA-512, LLH-768 with SHAKE256-768, and LLH-1024 with SHAKE256-1024. These comparisons reflect only software-efficiency differences under the same output length. Since the security strengths of these algorithms are not the same, the comparison is not cryptographically equivalent and should not be used to draw conclusions about the overall superiority of one algorithm over another.

6.1.2 ARM Platform

This section provides a software performance evaluation of the single-thread implementation and the multi-threaded parallel implementations for the four standard LLH instances LLH-256, LLH-512, LLH-768, and LLH-1024 (corresponding to word widths $w = 32, 64, 96, \text{ and } 128$, respectively). Unlike Section 6.1.1, which compares the reference and optimized implementations on a single server platform, the focus of this section is on the scalability and practical benefit of the parallelization strategies under different hardware environments. To this end, the tests are carried out simultaneously on a general-purpose desktop processor (Intel Core i7) and on an embedded platform built around an ARM architecture (Raspberry Pi), so as to quantify,

across two environments that differ markedly in compute power, cache capacity, and memory bandwidth, the throughput improvement and speedup brought by multi-threaded parallelism, and to analyze the impact of overheads such as thread creation, scheduling, and inter-core migration on the results.

To distinguish the different parallelization strategies, this section provides three implementation versions. The algorithm definition, interface semantics, length encoding, padding rules, and the absorbing and squeezing process of all three are completely consistent with the algorithm description; they differ only in task partitioning and thread organization, so their performance differences can be attributed directly to the parallel structure itself. The versions are described as follows:

- v1 (single-thread sequential implementation, `llh_hash_v1`): the baseline version, which completes all absorbing and squeezing operations sequentially within a single thread without any concurrency, and serves as the unified reference for measuring the speedup of each parallel version.
- v2 (multi-threaded parallel merging implementation, `ParallelHash`): drawing on the tree structure of `ParallelHash`, the input message is split into several blocks whose digests are computed in parallel by multiple threads, and the intermediate results are then merged into the final hash value.
- v3 (multi-threaded block-wise parallel merging implementation, K12-style): the computation is organized following the block-wise parallel structure of KangarooTwelve (K12), and on top of block-wise parallelism a dedicated processing path is provided for short messages to reduce the parallel overhead for small inputs.

Note: The number of threads in the multi-threaded versions is fixed at 4 on the local desktop platform, and is determined by the number of available cores on the Raspberry Pi platform.

6.1.2.1 Test Environment and Test Cases

To ensure that the test results are reproducible, this section runs the same test scripts independently on the local desktop platform and the Raspberry Pi platform, and within each platform the operating system, CPU, compiler version, compilation optimization options, and test framework are kept fixed, so as to ensure comparability between different implementation versions on the same platform. The hardware and software configurations of the two platforms are shown in Table 6-5, respectively.

Table 6-9. Local platform test environment

Item	Specification
Operating system	Linux Ubuntu 22.04
CPU	Intel(R) Core(TM) i7-12700
Compiler	GCC with -O2 optimization
Test framework	Python 3 + subprocess

Table 6-10. Raspberry Pi platform test environment

Item	Specification
Operating system	Raspberry Pi OS (Linux ARM)
CPU	ARM Cortex-A series, low-frequency multi-core
Compiler	GCC with -O2 optimization
Test framework	Python 3 + subprocess

The test cases are divided into two groups, defined in Table 6-11. The first group characterizes the fine-grained variation of performance with input length in the short-message range, covering every input length from 1B to 512B, where 512B equals 4096 bits, that is, 2^{12} bits, with a step of 1B. The second group (Test2) reflects the overall performance at typical message sizes,

taking the three data sizes of 1KB, 1MB, and 1GB. All of the above tests are run separately on the four standard instances LLH-256, LLH-512, LLH-768, and LLH-1024, and each data size is independently measured 100 times with the arithmetic mean taken as the final result, so as to cover the performance under different word widths and different parallel granularities.

Table 6-11. Raspberry Pi platform test environment

ID	Data size
Test1	1-512B, step 1B
Test2	1KB, 1MB, 1GB

6.1.2.2 Test Procedure

The local platform and the Raspberry Pi platform use the same test procedure, with the following steps:

- Generate random input data of the specified size.
- Run the v1 single-thread version and then the v2 and v3 multi-threaded versions in turn, starting the timer after the thread pool has warmed up.
- Record the full elapsed time of each run, including the data read time (read_time) and the computation time (compute_time).
- Repeat the measurement independently 100 times for each data size, and take the arithmetic mean as the final result for that group.

To ensure the validity and rigor of the test results, the variables are strictly controlled during testing. First, each comparison always uses the same platform, the same compiler version, and the same test framework, ensuring consistency of architecture and toolchain. Second, the number of threads in the multi-threaded versions is fixed, and run-time dynamic scheduling is not allowed, so as to avoid introducing additional random error. Third, the local platform and the Raspberry Pi platform run the same test scripts independently,

ensuring that the two environments are isolated from and do not interfere with each other.

6.1.2.3 Test Results

Based on the two types of hardware platforms, this section presents the test results from three perspectives: throughput, speedup, and per-instance speedup. The relevant performance metrics are defined as follows:

- Execution time: the time taken for the complete execution of the algorithm, equal to the sum of the data read time and the computation time ($\text{read_time} + \text{compute_time}$, in ms).
- Throughput: the amount of data processed per unit time, equal to the data size divided by the execution time (in MB/s).
- Speedup: the ratio of the execution time of the v1 baseline version to that of the parallel version, where a value greater than 1 indicates that the parallel implementation achieves acceleration relative to the baseline.
- Local platform results.

Figure 6-12 gives the fine-grained speedup curves by instance for Test1, where the data size ranges from 1B to 512B. ParallelHash has speedup consistently below 1. For small data sizes, the overhead of thread creation and management is much larger than the benefit of parallel computation. The K12-style version exhibits fluctuations, but its overall speedup is greater than 1, indicating that instruction-level parallelism has a positive effect on cryptographic speed. The instability of K12-style speedup has both algorithmic and hardware causes. At the algorithm level, small inputs may introduce extra scheduling, branching, or data-organization overhead, and may fail to trigger more efficient SIMD vectorized paths, resulting in local performance disadvantages. At the hardware level, the Alder Lake heterogeneous architecture of the test CPU, with P-cores and E-cores, further

amplifies the fluctuation. Small tasks run for a very short time, and thread scheduling, core migration, and cache state, such as L1/L2 hits, can significantly affect total execution time. If tail processing, branch misprediction, or execution on an efficiency core occurs, the overall performance may become lower than the baseline.

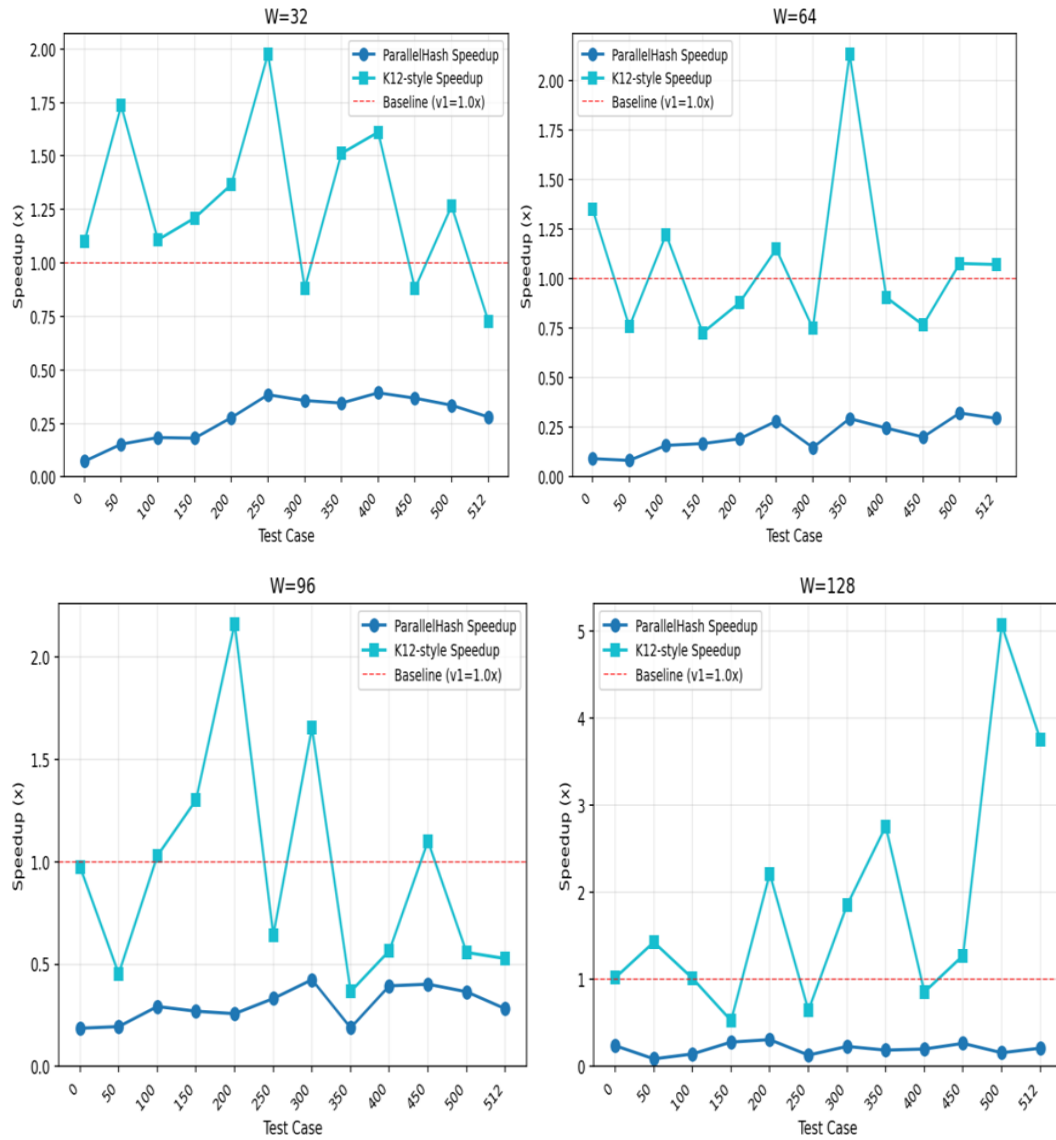


Figure 6-12. Local platform: fine-grained speedup curves for each standard instance under Test1 (1B-512B)

Figure 6-13 gives the throughput comparison of the four standard

instances under the three data sizes 1KB, 1MB, and 1GB. When the data size is 1KB or 1MB, the throughput of ParallelHash is lower than the v1 baseline. When the data size reaches 1GB, its throughput improves markedly and the speedup approaches the number of threads used. The main reason is again that, for small data, thread-creation overhead greatly exceeds the execution time of the cryptographic computation. In contrast, K12-style already shows clear acceleration at 1MB. As data size increases, fixed overheads such as parallel scheduling, thread creation, and cache cold start are significantly amortized. Overheads that dominate small data become almost negligible for GB-level input, allowing the structural advantage of K12 to be fully expressed.

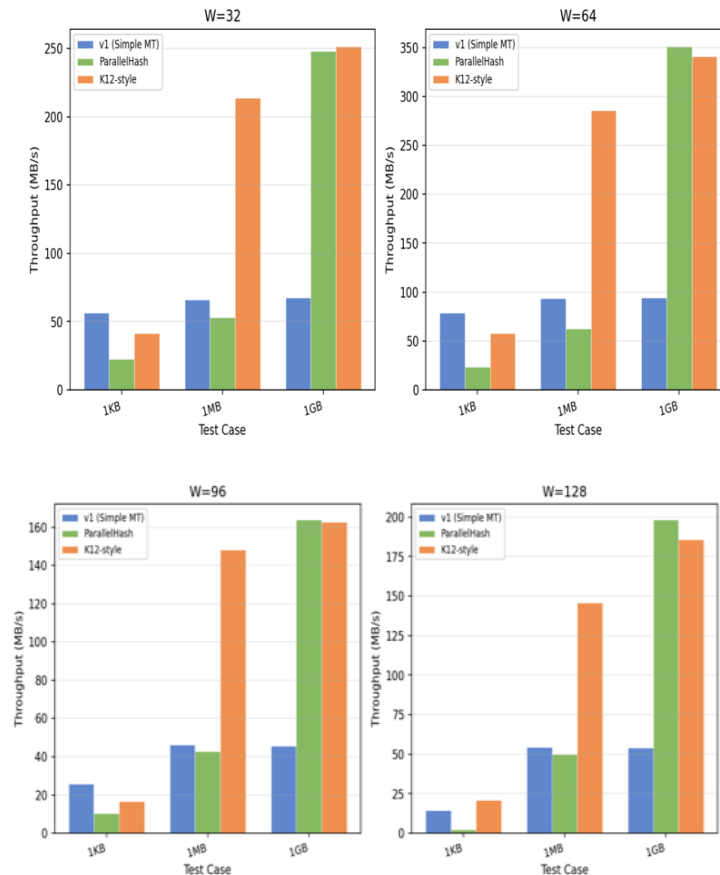


Figure 6-13. Local platform: throughput comparison of the three implementation versions under 1KB, 1MB, and 1GB

Figure 6-14 further uses speedup as the metric, giving the speedup of ParallelHash and K12-style relative to the v1 baseline on the local platform for the three data sizes of 1KB, 1MB, and 1GB, which can be cross-checked against the throughput results in Figure 6-13. Combining the measured data in Table 6-12 to Table 6-14 the speedup of both versions exhibits a clear evolution with data size. At 1KB, the speedup of ParallelHash is below 1 for every instance, and K12-style is also below 1 in most cases, indicating that the fixed overhead of thread creation and scheduling still exceeds the computation time saved by parallelism. As the data size grows to 1MB, ParallelHash remains below 1 because of merging and thread-management overhead, whereas K12-style is the first to achieve a clear speedup thanks to its streamlined block-wise parallel structure. When the data size increases further to 1GB, the fixed overhead of both versions is fully amortized, and their speedups converge to about 3.4x to 3.7x, approaching the practical upper bound attainable with the number of threads used on the local platform in the presence of scheduling overhead. Because the local platform has ample memory bandwidth and a multi-level cache, the system remains mainly compute-bound at large data sizes, so the performance advantage of the parallel structures is released steadily as the data size grows, which is fully consistent with the throughput trend reflected in Figure 6-13.

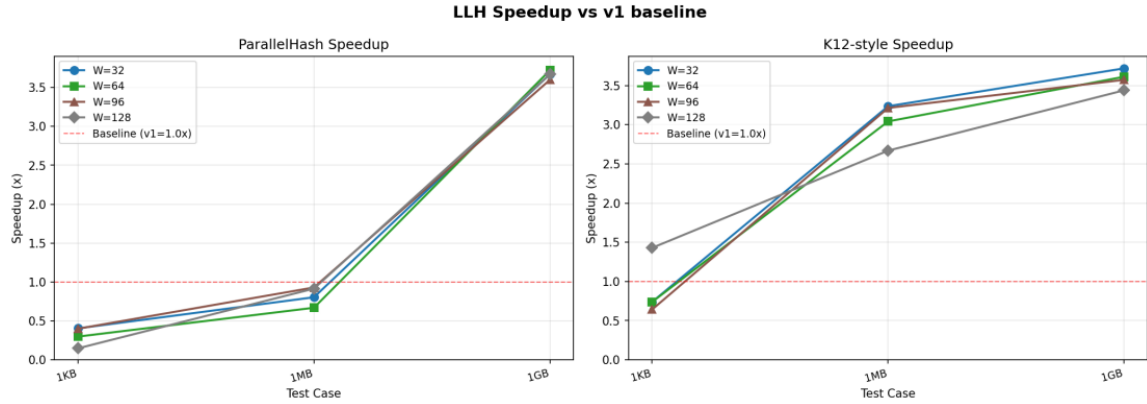


Figure 6-14. Local platform: speedup of ParallelHash and K12-style relative to the v1 baseline under 1KB, 1MB, and 1GB

- Raspberry Pi platform results.

The following three figures present the same metrics on the Raspberry Pi ARM platform, for comparison with the local platform results. Unlike the local desktop platform, which has ample memory bandwidth, the Raspberry Pi uses a low-frequency, multi-core Cortex-A series processor whose memory bandwidth and cache capacity are relatively limited. Its parallel behavior across different data sizes therefore differs markedly from that of the local platform, which makes it possible to examine the practical scalability of the parallelization strategies in a resource-constrained embedded environment.

Figure 6-15 gives the fine-grained speedup curves by instance for Test1 under the data-size range 1B-512B. The test shows that both ParallelHash and K12-style can obtain stable performance gains, but their acceleration characteristics are fundamentally different. ParallelHash has an average speedup of about 2 and shows stable gains insensitive to input size. K12 reaches about 35x speedup for extremely small inputs, but the speedup gradually decreases to about 2x as the data size increases. This indicates that, in a resource-constrained hardware environment with limited parallel capability, small-data performance is mainly determined by implementation-

path optimization rather than the parallel structure itself. The generic tree structure of ParallelHash is difficult to exploit for small inputs, so it mainly provides lightweight optimization gains. K12 obtains highly efficient execution for very small inputs through a specialized short-message path, but this advantage is gradually amortized as the computational load increases. Overall, this difference reflects two design orientations: ParallelHash emphasizes structural generality and scalability, while K12 emphasizes practical performance across scales, especially for short messages.

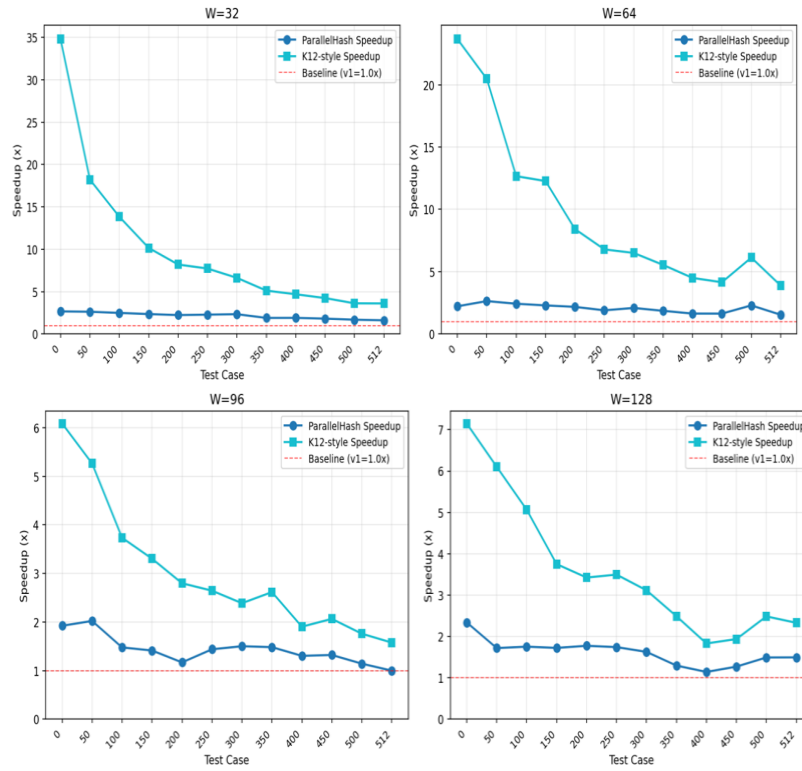
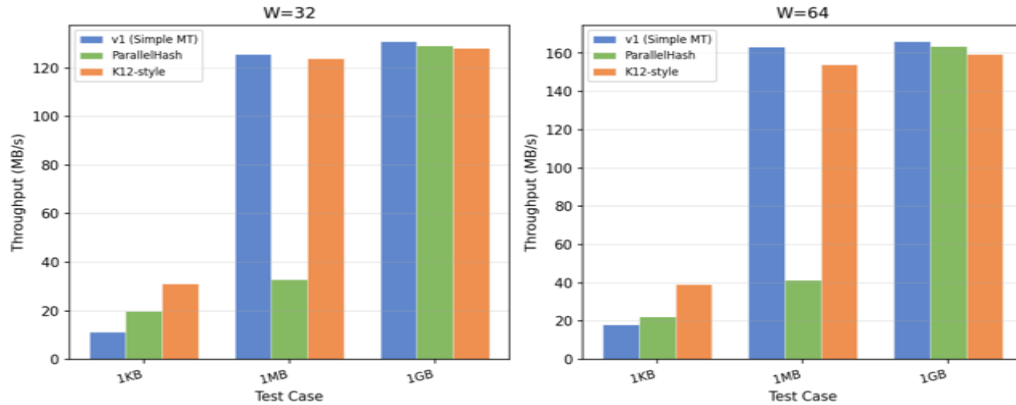


Figure 6-15. Raspberry Pi platform: fine-grained speedup curves for each standard instance under Test1 (1B-512B)

Figure 6-16 gives the throughput comparison by instance for the three data sizes 1KB, 1MB, and 1GB. When the data size increases to 1MB and 1GB, the speedups of both ParallelHash and K12-style gradually approach 1. This indicates that the performance advantage brought by parallel structure

largely disappears for large inputs. The system changes from compute-bound to memory-bound. In small-data ranges, performance is mainly determined by the algorithm path, function-call overhead, and instruction-level optimization, so differences between implementations are visible. When the input expands to MB or GB level, the computation cost of the internal round function is masked by large data accesses, and the bottleneck shifts to the memory subsystem. At this point, whether the implementation is serial or tree-parallel, it must continuously read data from memory and write state back. Overall throughput is limited by memory bandwidth and cache hierarchy rather than by the algorithm itself.

On the Raspberry Pi Cortex-A72 architecture, this phenomenon is more pronounced. Its memory bandwidth and cache capacity are relatively limited. Multi-core parallelism causes bandwidth contention when processing large streaming data, preventing all threads from running at full speed simultaneously. As a result, the theoretical acceleration brought by parallelization is completely offset by the memory-access bottleneck, causing throughput to converge and speedup to approach 1. This result contrasts clearly with the local server results.



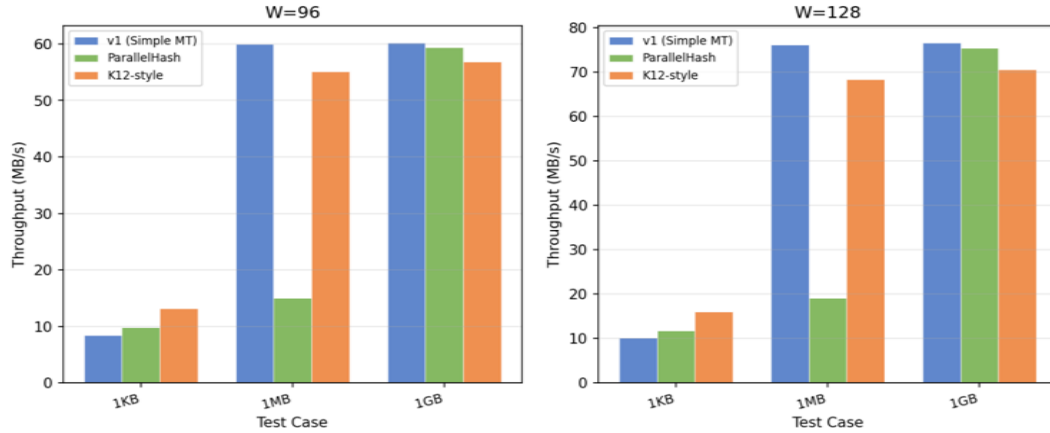


Figure 6-16. Raspberry Pi platform: throughput comparison of the three implementation versions under 1KB, 1MB, and 1GB

Figure 6-17 uses speedup as the metric, giving the speedup of ParallelHash and K12-style relative to the v1 baseline on the Raspberry Pi platform for the three data sizes of 1KB, 1MB, and 1GB, which can be cross-checked against the throughput results in Figure 6-2-5. The measured data in Tables 611 to 613 show that the speedup on the Raspberry Pi platform follows a non-monotonic pattern that is opposite to that of the local platform. At the small data size of 1KB, the speedups of both ParallelHash and K12-style are above 1 (ParallelHash about 1.15 to 1.76, K12-style about 1.55 to 2.75), indicating that on the resource-constrained ARM architecture the performance in the short-message range is governed mainly by implementation-path optimization, and both versions achieve a positive gain over the serial baseline. As the data size grows to 1MB, the speedup of both versions instead drops below 1: ParallelHash falls sharply to about 0.25, clearly worse than the v1 baseline, and K12-style also declines to about 0.90 to 0.99, showing that multi-threaded parallelism not only fails to accelerate but introduces extra cost from thread merging and data movement. When the data size increases further to 1GB, the speedup recovers somewhat, with ParallelHash returning to about

0.985 and K12-style to about 0.92 to 0.98, yet neither exceeds the baseline. This behavior is consistent with the throughput results in Figure 6-2-5: the Cortex-A series architecture used by the Raspberry Pi has relatively limited memory bandwidth and cache capacity, and as the input grows to the MB and even GB level the system shifts from compute-bound to memory-bandwidth-bound. Multi-core parallelism then causes bandwidth contention when processing large-scale streaming data, so the theoretical speedup of parallelization is cancelled by the memory-access bottleneck, and the speedup of both ParallelHash and K12-style can only approach but cannot stably exceed 1. This cross-platform comparison clearly shows that the practical benefit of a parallelization strategy depends heavily on the memory-subsystem characteristics of the underlying hardware.

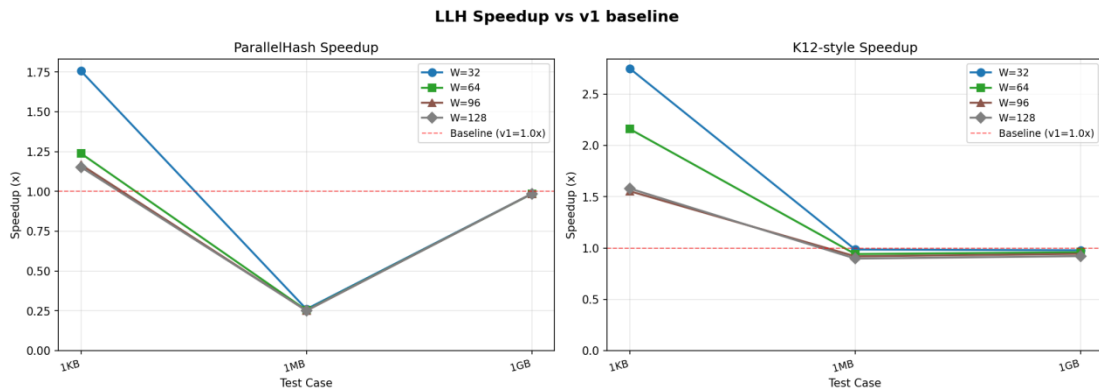


Figure 6-17. Raspberry Pi platform: speedup of ParallelHash and K12-style relative to the v1 baseline under 1KB, 1MB, and 1GB

6.1.2.4 Raw Data Summary

This subsection lists the raw measured data of the above tests. All values are averages over 100 repetitions per group. Execution time is given in milliseconds, and speedup is a dimensionless value, where a value greater than 1 indicates acceleration relative to the v1 single-thread baseline. To stay consistent with the instance naming used throughout this document, the

following tables identify the standard instances with word widths $w=32, 64, 96$, and 128 as LLH-256, LLH-512, LLH-768, and LLH-1024, respectively.

- Measured data for the three versions on the local platform

The following tables give the execution time and the respective speedups of the three implementation versions on the local platform (with the v1 speedup = 1.00 as the baseline; a PH or K12 speedup greater than 1 indicates better performance than v1).

Table 6-12. Data size: 1KB

Instance	v1(ms)	v1speedup	PH(ms)	PHspeedup	K12(ms)	K12speedup
LLH-256	0.017	1.00	0.043	0.404	0.023	0.739
LLH-512	0.012	1.00	0.042	0.298	0.017	0.738
LLH-768	0.038	1.00	0.095	0.399	0.059	0.642
LLH-1024	0.067	1.00	0.455	0.147	0.047	1.429

Table 6-13. Data size: 1MB

Instance	v1(ms)	v1speedup	PH(ms)	PHspeedup	K12(ms)	K12speedup
LLH-256	15.124	1.00	18.812	0.804	4.672	3.237
LLH-512	10.639	1.00	15.906	0.669	3.496	3.043
LLH-768	21.674	1.00	23.309	0.930	6.741	3.215
LLH-1024	18.319	1.00	20.030	0.915	6.862	2.670

Table 6-14. Data size: 1GB

Instance	v1 (ms)	v1 speedup	PH (ms)	PH speedup	K12(ms)	K12 speedup
LLH-256	15147.932	1.00	4125.093	3.672	4072.121	3.720
LLH-512	10859.831	1.00	2916.643	3.723	3006.068	3.613
LLH-768	22482.521	1.00	6245.006	3.600	6290.129	3.574
LLH-1024	18938.811	1.00	5160.865	3.670	5505.842	3.440

- Measured data for the three versions on the Raspberry Pi platform

The following tables give the same metrics on the Raspberry Pi platform, forming a cross-platform comparison with the local platform results.

Table 6-15. Data size: 1KB

Instance	v1 (ms)	v1 speedup	PH (ms)	PH speedup	K12 (ms)	K12 speedup
LLH-256	0.086	1.00	0.049	1.757	0.031	2.751
LLH-512	0.054	1.00	0.043	1.239	0.025	2.161
LLH-768	0.115	1.00	0.099	1.168	0.074	1.555
LLH-1024	0.096	1.00	0.083	1.153	0.061	1.581

Table 6-16. Data size: 1MB

Instance	v1(ms)	v1speedup	PH(ms)	PHspeedup	K12(ms)	K12 speedup
LLH-256	7.959	1.00	30.453	0.261	8.073	0.986
LLH-512	6.116	1.00	24.038	0.254	6.495	0.942
LLH-768	16.683	1.00	66.369	0.251	18.139	0.920
LLH-1024	13.145	1.00	52.217	0.252	14.637	0.898

Table 6-17. Data size: 1GB

Instance	v1 (ms)	v1 speedup	PH (ms)	PH speedup	K12 (ms)	speedup
LLH-256	7817.919	1.00	7928.892	0.986	7986.995	0.979
LLH-512	6160.203	1.00	6252.141	0.985	6420.032	0.960
LLH-768	17001.448	1.00	17247.321	0.986	18013.110	0.944
LLH-1024	13371.120	1.00	13568.132	0.985	14500.361	0.922

6.1.2.5 Result Analysis

Combining the measured results from the two types of platforms, the following analytical conclusions can be drawn.

- Significant parallel acceleration for large data (local platform): at the 1GB data size, the speedups of ParallelHash and K12-style relative to v1 are stably between about 3.4x and 3.7x and are consistent across the four standard instances, which shows that for CPU-intensive large-data hashing tasks, multi-threaded parallelism can bring clear and predictable performance gains.

- Parallel overhead offsets the benefit for small data (local platform): at the 1KB data size, the speedups of the parallel versions are generally below 1, as the fixed overhead of thread creation and scheduling exceeds the

computation time saved by parallel computation, so the multi-threaded versions are slower than the single-thread baseline.

- The Raspberry Pi platform shows the opposite pattern: in the small-data 1KB case, K12-style can still achieve a speedup of about 1.5x to 2.7x, but in the 1MB and 1GB cases the speedups of the parallel versions approach 1 or even fall slightly below the baseline, which indicates that the thread-scheduling overhead has become comparable to the computation load and that memory bandwidth has become the main factor limiting performance, with ParallelHash even slightly slower than v1 for large data. Overall, the benefit of parallelization depends heavily on the compute power and memory-access characteristics of the platform, and a suitable implementation version should be chosen according to the specific hardware environment and message size.

6.2 Hardware Performance Evaluation

6.2.1 Hardware Implementation

The hardware module of LLH consists of three main units: an AXI bus interface, a data-padding unit, and an absorbing-and-squeezing unit. The absorbing-and-squeezing unit is the core module. It is the key factor that determines the performance of the algorithm and is therefore the focus of the evaluation. The other two units still have substantial room for implementation optimization and are not deliberately emphasized in this evaluation.

It should be noted that LLH supports four different output-length modes. In the present implementation, these modes are selected by software configuration. As a result, the power consumption and implementation cost for shorter modes are higher than those of a circuit designed for a single output length. In practical applications, a dedicated implementation for a specific output length may be adopted to satisfy the corresponding requirements.

The AXI bus interface supports 128-bit data reads and writes. When the data width exceeds 128 bits, multiple bus operations are required.

The data-padding unit is designed according to the padding rule specified by the algorithm. Whenever the accumulated input data reaches the bitrate, one message block is output to the absorbing-and-squeezing unit. If the final group of input data is shorter than the bitrate, padding is applied to this group.

As the core computational component of the algorithm, the absorbing-and-squeezing unit implements the round function in combinational logic in order to reduce the implementation area and improve operating efficiency. A finite-state machine is used to control data transfer between rounds and the state transitions between the absorbing and squeezing phases.

According to the design principle of the algorithm, once the absorbing-and-squeezing module obtains the message length, it can start computing $P(S^{(0)})$ without waiting for the input data. Therefore, the input cost of the first data block can be hidden at the algorithmic level. This design minimizes the time consumed by data input and ensures that the overall computation does not require additional clock cycles.

6.2.2 Computational Efficiency

Algorithm	clock cycle count (cycles)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	33	81	273	529	2065	4113	8209	32785
LLH-512	49	73	217	409	1561	3097	6169	24601
LLH-768	65	97	225	385	1409	2785	5505	21889
LLH-1024	81	81	201	361	1321	2601	5161	20521

Algorithm	normalized performance (bit/cycle)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB

LLH-256	7.7576	12.6420	15.0037	15.4858	15.8683	15.9339	15.9669	15.9917
LLH-512	5.2245	14.0274	18.8756	20.0293	20.9917	21.1611	21.2469	21.3117
LLH-768	3.9385	10.5567	18.2044	21.2779	23.2562	23.5318	23.8096	23.9521
LLH-1024	3.1605	12.6420	20.3781	22.6925	24.8055	25.1965	25.3966	25.5489

When the input size reaches 1024 Bytes, the performance is already high and gradually approaches the theoretical peak performance of the algorithm.

6.2.3 Hardware Area (12nm)

The post-synthesis total area is as follows.

Module	Area (μm^2)
Configuration module	293.1794
Padding module	5960.3928
Absorbing-and-squeezing module	13768.1513
Total	20039.307572

The post-synthesis module areas are shown below.

Module	Total area (μm^2)	Percentage	Combinational logic area (μm^2)	Non-combinational logic area (μm^2)
Top-level module	20039.3076	100	17.5841	0
Configuration module	293.1794	1.5	189.6284	100.8968
Padding module	5960.3928	29.7	4977.5617	979.2922
Absorbing-and-squeezing module	13768.1513	68.7	8309.7724	4349.5835

The area of the minimum-size NAND2 gate in the process library is $0.11059\mu\text{m}^2$. This value is used as the normalization parameter. The resulting normalized areas are as follows.

Module	NAND2-equivalent gate count
Configuration module	2651.0480
Padding module	53896.3089
Absorbing-and-squeezing module	124497.2538

Total	181203.6131
-------	-------------

6.2.4 Hardware Latency (12nm)

The critical path of the algorithm circuit is the path from the internal state through the permutation function to the updated internal state. According to the synthesis results, the maximum delay of this path is 0.64 ns. Therefore, the maximum operating frequency that satisfies STA closure is 1.56 GHz.

6.2.5 Throughput

Algorithm	throughput (Mb/s)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	12121.21	19753.08	23443.22	24196.60	24794.19	24896.67	24948.23	24987.04
LLH-512	8163.27	21917.81	29493.09	31295.84	32799.49	33064.26	33198.25	33299.46
LLH-768	6153.85	16494.85	28444.44	33246.75	36337.83	36768.40	37202.54	37425.19
LLH-1024	4938.27	19753.09	31840.80	35457.06	38758.52	39369.47	39682.23	39920.08

The table above reports simulation results at a clock frequency of 1.56 GHz for different input sizes. As the input size increases, the overall throughput of the module continues to increase. When the input size exceeds 8192 Bytes, the growth of throughput becomes slower and the performance approaches the theoretical peak of the algorithm.

6.2.6 Normalized Throughput-to-Area Ratio

The area of the minimum-size NAND2 gate in the process library is 0.11059 μ m², which is used as the normalization parameter. The normalized throughput-to-area ratio is computed as:

$$\begin{aligned}
 & \text{Normalized throughput – to – area ratio} \\
 &= \frac{\text{operating throughput}}{\text{post – synthesis total area/NAND2 gate area}}.
 \end{aligned}$$

The normalized throughput-to-area ratios for different input sizes are shown below.

Algorithm	normalized throughput-to-area ratios (Mb/s/NAND2 equivalent gate count)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	0.0669	0.1090	0.1294	0.1335	0.1368	0.1374	0.1377	0.1379
LLH-512	0.0451	0.1210	0.1628	0.1727	0.1810	0.1825	0.1832	0.1838
LLH-768	0.0340	0.0910	0.1570	0.1835	0.2005	0.2029	0.2053	0.2065
LLH-1024	0.0273	0.1090	0.1757	0.1957	0.2139	0.2173	0.2190	0.2203

6.2.7 Power Consumption

The idle power consumption is as follows.

Module	LLH-256/512/768/1024
Configuration module	47.03 uW
Padding module	72.48 uW
Absorbing-and-squeezing module	1.73 mW
Total	1.85 mW

The full-load power consumption is as follows.

Module	LLH-256	LLH-512	LLH-768	LLH-1024
Configuration module	80.77 uW	95.59 uW	99.94 uW	779.82 uW
Padding module	848.29 uW	1.57 mW	2.67 mW	2.57 mW
Absorbing-and-squeezing module	54.32 mW	59.22 mW	63.88 mW	67.16 mW
Total	55.25 mW	60.88 mW	66.65 mW	70.51 mW

The total power consumption is as follows.

Module	LLH-256	LLH-512	LLH-768	LLH-1024
Configuration module	127.8 uW	142.62 uW	146.97 uW	826.85 uW
Padding module	920.77 uW	1.64mW	2.74 mW	2.64 mW
Absorbing-and-squeezing module	56.05 mW	60.95 mW	65.61 mW	68.89 mW
Total	57.1 mW	62.73 mW	68.5 mW	72.36 mW

Algorithm	power-normalized throughput (Mb/s/mW)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	88.22	143.77	170.62	176.11	180.46	181.20	181.58	181.86
LLH-512	54.08	145.20	195.39	207.33	217.30	219.05	219.94	220.61
LLH-768	37.33	100.07	172.57	201.71	220.46	223.07	225.71	227.06
LLH-1024	28.36	113.45	182.87	203.64	222.60	226.11	227.91	229.27

6.2.8 Performance Analysis

At the algorithm-performance level, the current implementation uses a 128-bit AXI interface, resulting in relatively low bus utilization. Bus utilization can be improved by supporting AXI Burst transfers and adding write and read FIFOs. This allows multiple data beats to be transferred continuously after a single address handshake, thereby effectively improving bus utilization and reducing interface overhead.

At the implementation-performance level, the design of the round function reflects the low-latency objective of the algorithm. The synthesis results above also show that the critical path of the round function is short. When the target frequency is not excessively high, several rounds of the round function may be unrolled to reduce the actual number of operating cycles. The main cost of this optimization is the increase in area and design complexity caused by the unrolled logic. However, the operating frequency usually does not decrease linearly after implementation optimization, so this approach can still provide performance benefits.

At the algorithm-configuration level, LLH supports four different output-length modes. In practical applications, implementations can be optimized for a specific output length to achieve a better balance among resource usage, operating frequency, efficiency, and power consumption.

7 Algorithmic Innovation and Advantages

7.1 Algorithmic Innovation

LLH is a family of ultra-low latency cryptographic hash algorithms based on a variant of the sponge construction. The key contributions are summarized as follows:

A breakthrough in the construction theory of large-scale, high-diffusion, and low-latency nonlinear permutations. By generalizing cyclic-circuit functions, we established a construction method for nonlinear permutations that simultaneously optimizes for large scale, high diffusion, and minimum latency (refer to **Theorem 2-1**). This provides a theoretical foundation for designing ultra-low latency cryptographic algorithms.

The ultra-low latency hash algorithm. The exceptional low-latency performance of LLH stems from its structural selection and the design of basic components. At the structural level, the round function of the permutation P is built using bit-slicing. In this construction, the circuit delay of linear expansion in the row direction and round-constant addition can be regarded as negligible; the main latency is primarily determined by the transformations in the slice direction. In the slice direction, an SPN structure decomposes the operation into three smaller components: small S-box transformation, derangement permutation, and row-wise linear transformation. The 6-bit small S-box reaches the mathematical lower bound for nonlinear permutation latency. The derangement permutation has no delay in hardware implementation. The row-wise linear transformation also reaches the mathematical lower bound for latency while attaining the optimal branch number. Consequently, the circuit depth of the whole round function of P is only 4, which is nearly optimal for diffusion branch number 5. This

construction gives LLH extremely low circuit implementation latency, making it the lowest-latency hash algorithm known to date.

The feed-forward sponge variant. It encodes the message length into the initial state. This not only inherits the security advantages of the original sponge construction, but also strengthens its resistance against length-extension attacks.

7.2 Algorithmic Advantages

Compared with known hash standards such as SM2, SHA-2, and SHA-3, LLH has the following advantages.

- LLH has extremely low hardware implementation latency and is among the lowest-latency known hash algorithms.
- LLH adopts a feed-forward sponge-variant design and encodes the message length into the initial state. It inherits the security advantages of the sponge construction, and further improves resistance against length-extension attacks.
- LLH primarily uses implementation-friendly basic operations such as XOR, logical AND, and cyclic shift. Therefore it incurs very low software and hardware implementation costs while delivering exceptional efficiency. Furthermore, its low-depth construction is highly suitable for acceleration using SIMD and other vector instructions, allowing various optimization strategies to meet the requirements of diverse platforms.
- LLH has superior resistance against side-channel attacks. Without incorporating any side-channel protection mechanisms in the hardware design, LLH can theoretically reach the minimum leakage level against the strongest side-channel attacks considered in this report.

In summary, LLH mainly pursues extreme low latency and achieves an excellent balance among security, implementation, software and hardware

cost, and efficiency. This balance represents the core value of LLH as a leading candidate for an ultra-low latency cryptographic hash algorithm.

8 Conclusion

LLH is a family of cryptographic hash algorithms characterized by ultra-low latency. It adopts a variant of the feedforward sponge construction and supports four output lengths: 256, 512, 768, and 1024 bits. The overall design is guided by three primary objectives: low latency, ease of implementation, and analyzability. Structurally, it employs a unified and regular sliced iterative architecture; at the component level, it uses nonlinear and linear primitives that achieve theoretical lower bounds; and at the round-function level, it realizes coordinated propagation of nonlinearity and diffusion through column substitution, row shifting, and constant addition.

From a design perspective, the most prominent feature of LLH is that its low-latency advantage is supported by a clear theoretical foundation. The basic 6-bit S-box and 4-bit linear transformation both achieve their respective theoretical lower bounds, and the resulting 24-bit super S-box and round function preserve a traceable low-depth structure. Therefore, the “ultra-low latency” property of LLH is not merely an empirical implementation optimization, but rather a theoretically grounded design outcome built from the ground up. Combined with the software and hardware performance evaluation in Chapter 6, LLH demonstrates clear structural advantages in latency-sensitive application scenarios.

From the security analysis perspective, LLH has established a relatively comprehensive layered evaluation framework. The key properties of the basic components, super S-box, round function, and overall mode have been systematically analyzed. In terms of classical security, security bounds for

reduced-round instances have been provided. In the context of post-quantum security, the one-wayness at the mode level can be reduced to the a single-iteration inversion problem over a fixed public capacity part in the quantum random permutation model. At the instance level, no structured full-round attack has been identified that compromises the post-quantum security goals under standard parameters. Therefore, we conclude that the security boundary of LLH is, within the current scope of research, generally clear and well-understood.

From an application perspective, the regular structure of LLH and its implementation friendliness make it suitable for both software SIMD environments and hardware platforms sensitive to combinational logic depth and gate-level latency. For resource-constrained devices, ultra-low latency authentication scenarios, and systems where hash computation delay is critical, LLH shows strong application potential.

In summary, LLH achieves a strong balance among design philosophy, cryptographic component selection, overall structure, security analysis, and implementation performance. Its main innovation lies in establishing a design paradigm centered on “provably minimal latency,” while simultaneously maintaining diffusion capability, structural transparency, and security analyzability. Based on current theoretical analysis and experimental evaluation, LLH can be regarded as a class of ultra-low latency cryptographic hash functions with distinctive characteristics and strong potential for further research and practical deployment.

Acknowledgement

We gratefully acknowledge the contributions of Dr. Liu Junrui, Ph.D. students Xu Rui, Ma Ruowei, Chen Xin, and Zhang Jianing, as well as master's students Su Jiaqi, Luo Lan, Tong Jingcheng, and Wang Chuangang, for their involvement in the design, implementation, and performance evaluation of the LLH algorithm. Their insightful suggestions and unwavering support during algorithm discussions, experimental validation, and manuscript refinement have been invaluable. We are deeply thankful for their assistance, which has been essential to the successful completion of this report.

References

- [1] Bertoni G, Daemen J, Peeters M, et al. Sponge functions[C]//ECRYPT hash workshop. 2007, 2007(9).
- [2] Chen S, Xiang Z, Zeng X, Zhang S: On the Relation-ships between Different Methods for Degree Evaluation. IACR Transactions on Symmetric Cryptology, 2021(1), 411-442. <https://doi.org/10.46586/tosc.v2021.i1.411-442>
- [3] Bertoni G, Daemen J, Peeters M, et al. On the indifferentiability of the sponge construction[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008: 181-197.
- [4] Damgård I B. A design principle for hash functions[C]//Conference on the Theory and Application of Cryptology. New York, NY: Springer New York, 1989: 416-427.
- [5] Rogaway P, Shrimpton T. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance[C]//International workshop on fast software encryption. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004: 371-388.
- [6] Mouha N. Automated techniques for hash function and block cipher cryptanalysis[J]. Belgium: Khatolieke Universiteit Leuven, 2012.
- [7] Mironov I, Zhang L. Applications of SAT solvers to cryptanalysis of hash functions[C]//International Conference on Theory and Applications of Satisfiability Testing. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006: 102-115.

- [8] Kocher P, Jaffe J, Jun B. Differential power analysis[C]//Annual international cryptology conference. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999: 388-397.
- [9] Brier E, Clavier C, Olivier F. Correlation power analysis with a leakage model[C]//International workshop on cryptographic hardware and embedded systems. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004: 16-29.
- [10] Bogdanov A, Kizhvatov I, Manzoor K, et al. Fast and memory-efficient key recovery in side-channel attacks[C]//International Conference on Selected Areas in Cryptography. Cham: Springer International Publishing, 2015: 310-327.
- [11] Grover L K. A fast quantum mechanical algorithm for database search[C]//Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
- [12] Bennett C H, Bernstein E, Brassard G, et al. Strengths and weaknesses of quantum computing[J]. SIAM journal on Computing, 1997, 26(5): 1510-1523.
- [13] Amy M, Di Matteo O, Gheorghiu V, et al. Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3[C]//International Conference on Selected Areas in Cryptography. Cham: Springer International Publishing, 2016: 317-337.
- [14] Feng X, Wang Q, Zhang A, Jiang Y: On the construction of ultra-low latency nonlinear permutation. preprint.